Università degli Studi di Milano
Facoltà di Scienze e Tecnologie
Corso di Laurea Magistrale in Informatica

IMPROVING BIOMEDICAL ENTITY LINKING VIA CONTEXT-AWARE DENSE RETRIEVAL

Relatore: Prof. Marco Mesiti

Correlatore: Dr. Darya Shlyk

Tesi di:
Antoin Sader
Matricola: 20320A

Anno Accademico 2024-2025

Abstract

Biomedical Entity Linking (BEL) is a fundamental task in biomedical text mining, responsible for grounding entity mentions appearing in unstructured text to canonical concepts in standardized Knowledge Bases (KBs) such as the Unified Medical Language System (UMLS). However, the increasing volume of biomedical literature and the high degree of lexical ambiguity in medical terminologies present significant challenges for current systems. Existing state-of-the-art name-based architectures, such as BioSyn, have achieved remarkable results using sparse and dense retrieval but suffer from two critical limitations: they are computationally inefficient when scaling to large dictionaries—often leading to Out-Of-Memory (OOM) errors—and they encode mentions in isolation, effectively discarding the semantic context required to resolve polysemous terms.

To address these challenges, we propose a scalable and context-aware dense retrieval framework that enhances the baseline BioSyn architecture. Our approach introduces two major architectural advancements. First, we transition from a mention-only embedding strategy to a context-aware approach, representing queries as full sentences annotated with special boundary tokens ([MS]/[ME]). This allows the model to leverage local semantic cues for resolving hard negatives. Second, we integrate Facebook AI Similarity Search (FAISS) to replace brute-force matrix operations, enabling efficient candidate generation and hard negative mining on massive ontologies without computational bottlenecks.

We evaluated our approach on both small-scale (NCBI Disease) and large-scale (MedMentions/UMLS) benchmarks. Experimental results demonstrate that the proposed method not only resolves the scalability issues of previous systems—reducing

training time by over 300%—but also achieves superior ranking performance. Specifically, our optimal combined embedding strategy, significantly improves Top-1 Accuracy and Mean Reciprocal Rank (MRR) compared to the BioSyn baseline. These findings suggest that our framework offers a robust and scalable solution for high-precision entity linking in complex biomedical texts

Contents

1 Introduction **6**
1.1 BioMedical Entity Linking 6
1.2 Limitations 7
1.3 Research Objectives 8

2 Related work **11**
2.1 Evolution of Biomedical Entity Representations 11
2.2 Biomedical Entity Representations with Synonym Marginalization (BioSyn) 12
2.3 Biomedical Entity Linking with Homonym Disambiguation (BELHD) 18
2.4 Comparative Analysis of Embedding Strategies 19

3 Methodology **23**
3.1 Training Pipeline 23
3.2 Evaluation pipeline 40

4 Experiments and Results **44**
4.1 Datasets used 44
4.2 Small Scale Results (NCBI Disease) 47
4.3 Large Scale Results (MedMentions & UMLS) 54
4.4 Results Conclusion 60

5 Conclusion **63**
5.1 Future Directions 63
5.2 Code Availability 64

List of Figures

1 Schematic Overview of Name-Based Biomedical Entity Linking Architectures 7
2 Comparative Visualization of Embedding Strategies: BioSyn vs. the Proposed Context-Aware Approach 10
3 BioSyn Architecture 12
4 Comparison of Input Representations: BioSyn, BELHD, and the Proposed Combined Model 22
5 Training Data Sources: Document Text (.txt) and Corresponding Concept Mentions (.concept) 24
6 Sample entries from the Knowledge Base Dictionary Showing Concept Unique Identifiers (CUIs) and Entity Names 28
7 Real-Time Training Log Displaying Training Metrics (Loss, Margin, and MRR) 40
8 Comparison of Training Duration across Experimental Configurations 48
9 Training Loss Progression on the Small Scale Datasets (NCBI Disease) 49
10 Training Margin progression for the experiments on the Small Scale Datasets (NCBI Disease). 51
11 Comparative Evaluation of Top-k Accuracy Metrics on the small scale dataset (NCBI Disease) 52
12 Training Loss Progression for the experiments on the Large Scale Datasets (MedMentions UMLS). 55

13 Training Margin progression for the experiments on the Large Scale Datasets (MedMentions UMLS). 57
14 Comparative Evaluation of Top-k Accuracy Metrics on the Large Scale Dataset (MedMentions UMLS) 58

List of Tables

1 Training Loss Averages across epochs for the experiments on the Small Scale Dataset (NCBI Disease) 49
2 Evaluation Results of Top-k Accuracy Metrics on the Small Scale Dataset (NCBI Disease) 53
3 Training Loss Averages across epochs for the experiments on the Large Scale Dataset (MedMentions UMLS) 55
4 Evaluation Results of Top-k Accuracy Metrics on the Large Scale Dataset (MedMentions UMLS) 59

List of Acronyms

Acc@k Accuracy at rank k

AMP Automatic Mixed Precision

BEL Biomedical Entity Linking

BELHD Biomedical Entity Linking with Homonym Disambiguation

BERT Bidirectional Encoder Representations from Transformers

BioNLP Biomedical Natural Language Processing

BioSyn Biomedical Entity Representations with Synonym Marginalization

CNN Convolutional Neural Network

CUI Concept Unique Identifier

FAISS Facebook AI Similarity Search

FP16 Half-Precision Floating Point

HNSW Hierarchical Navigable Small World

IVF Inverted File System

KB Knowledge Base

LLM Large Language Model

LSTM Long Short-Term Memory

MRR Mean Reciprocal Rank

NCBI National Center for Biotechnology Information

NER Named Entity Recognition

NLL Negative Log-Likelihood

OOM Out-Of-Memory

PQ Product Quantization

TF-IDF Term Frequency-Inverse Document Frequency

Chapter 1

Introduction

1.1 BioMedical Entity Linking

The exponential growth of biomedical literature has created a need for automated systems capable of extracting structured knowledge from unstructured text.
Biomedical Entity Linking (BEL), also known as Named Entity Normalization (NEN), is a fundamental task in BioNLP (BioMedical Natural Language Processing). It is defined as the process of mapping entity mentions identified in a document to their corresponding synonyms in a standardized Knowledge Base (KB).

BEL serves as a prerequisite for various downstream tasks, including Information Retrieval, Question Answering, and Drug Discovery. However, identifying the correct concept is non-trivial. Unlike general domain entity linking, the biomedical domain is characterized by a high degree of lexical variability and the need for domain-specific knowledge.

Current state-of-the-art approaches generally utilize Deep Learning and can be categorized into two main groups:

1. **Entity-based methods:** These approaches construct vector representations for KB concepts (nodes) often by aggregating information from the concept's neighbors, descriptions, or synonyms. The linking decision is performed by matching the mention embedding to these node embeddings [7, 8, 9].

2. **Name-based methods:** These approaches directly identify the best matching

surface form (name) in the KB. The model treats every synonym in the dictionary as a separate entry, embeds both query mention and dictionary names into a shared vector space, and calculates similarity via dense retrieval (Biosyn) or autogressive generation (GenBioEL [10]).

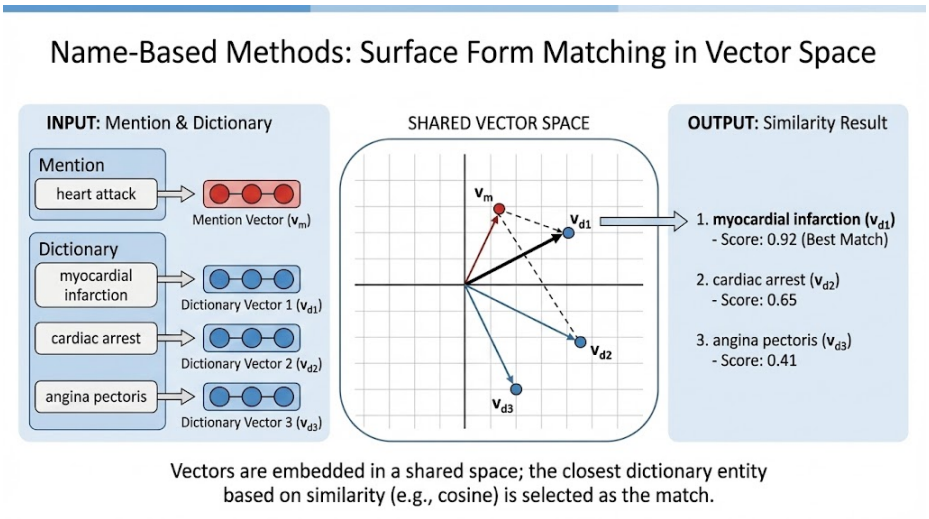

Figure 1: Schematic Overview of Name-Based Biomedical Entity Linking Architectures

The thesis builds upon the name-based method, specifically the BioSyn framework [1]. While BioSyn is highly effective at capturing morphological variations, their implementation failed to (1) scale to massive dictionaries and (2) capture the semantic context surrounding the mention.

1.2 Limitations

Normalization of biomedical entities is challenging due to the incompleteness of provided synonyms in standard KBs and the complex linguistic characteristics of medical text.

The primary challenges can be categorized as follows:

Lexical Variability (Synonymy)

A single biomedical concept can be referred to by numerous distinct surface forms, including acronyms, morphological variations, and descriptive phrases. A basic string-matching system will fail to link these variations.

Example: The terms "Motrin", "Advil", and "Ibuprofen" all refer to the same concept ID (`MeSH:D007052`), despite having completely different surface forms.

Hard Negatives

Conversely, mentions with high lexical overlap (similar spelling) may refer to entirely different concepts. Distinguishing between these "hard negatives" requires precise representation learning.

Example: "Dystrophinopathy" (`MeSH:D009136`) and "Bestrophinopathy" (`MeSH:C567518`) share a long suffix and high character n-gram overlap, yet they are distinct medical conditions. Standard sparse retrievers (like TF-IDF) often struggle to differentiate those as different entities.

Contextual Ambiguity

Perhaps the most critical limitation of existing name-based systems (like BioSyn) is their inability to handle ambiguity. Many mentions are polysemous, meaning that their interpretation depends entirely on the surrounding context which those systems are ignoring.

Example: The mention "Cold" could refer to the *Common Cold* (Virus), a *Low Temperature* (Environment), or *Chronic Obstructive Lung Disease* (Acronym).

Existing systems that embed only the mention string (e.g., encoding just "Cold") produce a static vector that cannot differentiate these meanings.

1.3 Research Objectives

This thesis aims to address the limitations of current name-based entity linking systems by proposing a scalable, context-aware dense retrieval framework. Our specific objectives are as follows:

1. **Scalable Retrieval via FAISS:** To eliminate the memory bottlenecks of the BioSyn baseline by integrating Facebook AI Similarity Search (FAISS), enabling efficient candidate generation for massive ontologies (e.g., UMLS [12] with 4M+ concepts).

2. **Context-Aware Representation:** To transition from a mention-only embedding strategy to a context-aware approach that integrates local sentence context. This involves representing queries as full sentences annotated with special boundary tokens (`[MS]` and `[ME]`) to resolve semantic ambiguity.

3. **End-to-End Dense Retrieval:** To replace sparse retrieval methods entirely with a robust dense retrieval mechanism that captures both morphological and semantic similarities.

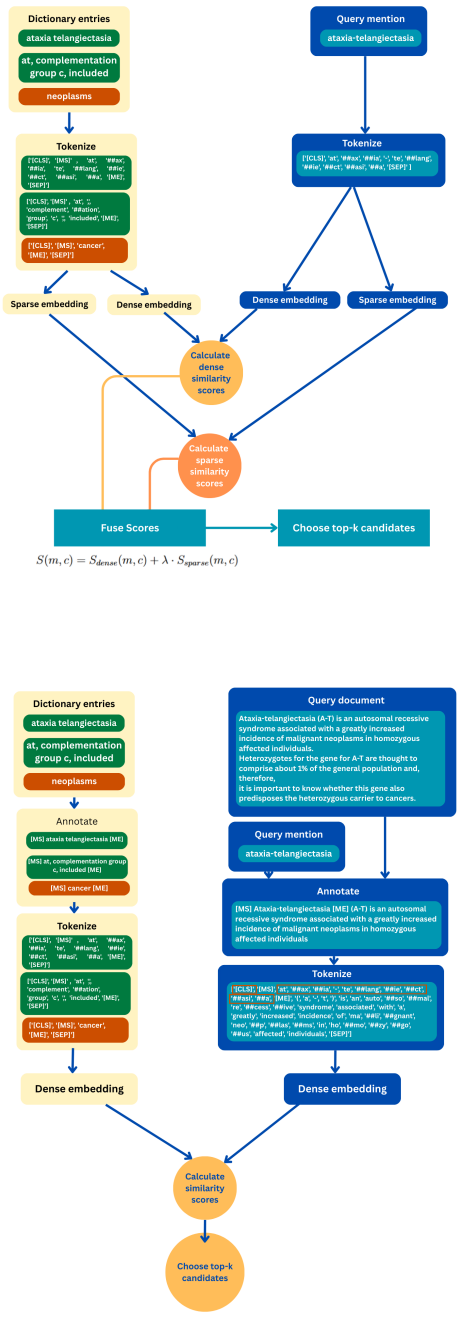

Figure 2: Comparative Visualization of Embedding Strategies: BioSyn vs. the Proposed Context-Aware Approach

Chapter 2

Related work

2.1 Evolution of Biomedical Entity Representations

Biomedical entity representations have historically relied on general-domain word representations adapted for the medical context.
Following the introduction of Word2Vec, biomedical versions of these static embeddings were widely adopted for various BioNLP tasks, including the biomedical normalization task.
The landscape shifted significantly with the introduction of **BioBERT**, a contextualized word representation model pre-trained on massive biomedical corpora (PubMed and PMC) using the BERT architecture.
BioBERT has since become the standard backbone for learning biomedical representations, including the BioSyn framework which is the baseline of this thesis.
Early machine learning approaches to normalization include **DNorm** [5], which utilized TF-IDF vectors as sparse representations in a pairwise learning-to-rank framework. Subsequent studies explored Convolutional Neural Networks (CNNs) for ranking candidates.
More recent architectures such as NSEEN and BNE map mentions and concept names into a shared latent space using LSTM models, refining embeddings via negative sampling techniques.

2.2 Biomedical Entity Representations with Synonym Marginalization (BioSyn)

BioSyn (Biomedical Entity Representations with Synonym Marginalization) [1], proposed by Sung et al. (2020), represents the current state-of-the-art in name-based biomedical entity linking.
Unlike preceding approaches that treated linking as a binary classification task (matching a mention to a single gold entity), BioSyn conceptualizes the problem as an iterative retrieval task optimized via Synonym Marginalization.

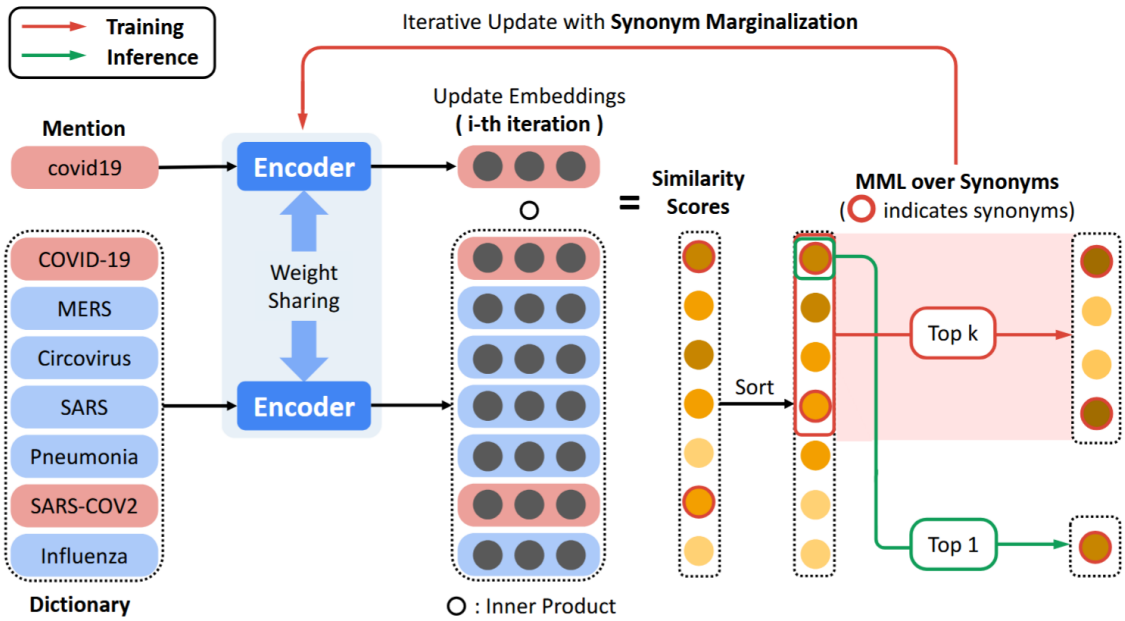

Figure 3: BioSyn Architecture

2.2.1 Overview of the Approach

The core philosophy of BioSyn is that an entity concept is defined by the set of all its synonymous surface forms.
During training, rather than forcing the model to distinguish between synonymous names (e.g., "Advil" vs. "Motrin"), the model maximizes the marginal probability

of retrieving *any* synonym belonging to the correct Concept Unique Identifier (CUI). This allows the model to learn a robust embedding space where morphologically diverse mentions of the same concept are embedded near to each other.

2.2.2 Embedding Strategy

The core objective of the BioSyn framework is to retrieve the top-k most relevant candidates from the dictionary for a given input mention m. To achieve this, the model calculates a similarity score $S(m, c)$ between the mention and every synonym c in the dictionary.

This scoring mechanism is designed to capture both morphological exactness and semantic meaning by combining two distinct scoring components: a **Sparse Score** (S_{sparse}) and a **Dense Score** (S_{dense}). The final ranking is determined by the weighted sum of these two scores.

1. Sparse Score Calculation (Lexical Similarity)

The sparse component handles the "lexical gap," addressing issues such as typos, suffix variations, and morphological differences common in biomedical text. Unlike word-level models that might treat rare drug names as "unknown" tokens, BioSyn utilizes a character-level representation to ensure every string has a unique footprint.

Specifically, the model constructs a vocabulary V consisting of character-level **uni-grams and bi-grams** derived from the dictionary concepts.

- **Vector Construction:** A sparse vector $E_{sparse}(m)$ is generated for the mention m using Term Frequency-Inverse Document Frequency (TF-IDF). The weight for the i-th n-gram is calculated as:

$$w_i = \mathrm{TF}(t_i, m) \times \mathrm{IDF}(t_i, \mathcal{D}) \tag{1}$$

  where $\mathrm{TF}(t_i, m)$ is the frequency of n-gram t_i in the mention, and $\mathrm{IDF}(t_i, \mathcal{D})$ is the inverse document frequency computed over the entire set of synonyms in the dictionary $\mathcal{D}$.

- **Score Calculation:** The sparse similarity score is computed as the inner product of the sparse vectors for the mention m and a candidate c:

$$S_{sparse}(m, c) = E_{sparse}(m)^\top E_{sparse}(c) \tag{2}$$

This score acts as a "morphological anchor," ensuring that candidates with high surface-form similarity are ranked highly.

2. Dense Score Calculation (Semantic Similarity)

To capture high-level semantic relationships (e.g., linking "heart attack" to "myocardial infarction"), BioSyn utilizes a dense encoder based on BioBERT.

- **Encoding:** The input is the raw mention string m. It is tokenized and passed through the pre-trained BioBERT encoder.
- **Representation:** The dense vector representation $E_{dense}(m)$ is derived from the final hidden state of the special `[CLS]` token, which captures the aggregate semantic meaning of the input:

$$E_{dense}(m) = \text{BioBERT}(m)_{\texttt{[CLS]}} \tag{3}$$

- **Score Calculation:** The dense similarity score is computed as the inner product of the dense vectors:

$$S_{dense}(m, c) = E_{dense}(m)^\top E_{dense}(c) \tag{4}$$

  Both the query mention and the dictionary candidates share the same encoder weights to map them into a shared vector space.

3. Final Score and Candidate Selection

The final similarity score $S(m, c)$ is a linear combination of the lexical and semantic views. A trainable scalar parameter λ is introduced to balance the contribution of the sparse signal against the dense signal:

$$S(m, c) = S_{dense}(m, c) + \lambda \cdot S_{sparse}(m, c) \tag{5}$$

For a given mention m, this score is calculated against all N entries in the dictionary. The top-k candidates with the highest $S(m, c)$ are then selected for the optimization phase.

2.2.3 Synonym Marginalization

The training phase of BioSyn is designed to optimize the encoder's parameters so that the similarity scores between a mention and all its valid synonyms are maximized. This process is driven by an iterative training loop that refines the vector space using a specialized loss function.

Training Loop and Candidate Retrieval

BioSyn operates on an iterative candidate retrieval mechanism. Unlike static classification tasks, the set of negative candidates evolves during training.

1. **Forward Pass:** For a given training mention m, the model computes the similarity scores $S(m, c)$ against the dictionary candidates.

2. **Candidate Selection:** It retrieves the top-k candidates. This set C_m typically includes the ground-truth synonym (positive) and several high-scoring incorrect entities (hard negatives).

3. **Score Calculation:** The similarity scores for these top-k candidates are normalized into a probability distribution.

Synonym Marginalization (Marginal NLL)

Standard contrastive learning approaches (like InfoNCE) typically assume a single positive target per query. However, biomedical concepts often have multiple valid synonyms in the dictionary (e.g., "renal failure" and "kidney failure" share the same CUI). Forcing the model to select only one specific synonym as the "correct" target would be semantically arbitrary and could penalize valid semantic matches.

To address this, BioSyn introduces **Synonym Marginalization**. Instead of maximizing the probability of a single target synonym, the objective is to maximize the marginal probability of the *set* of all valid synonyms present in the candidate pool.

Let $C_m = \{c_1, \ldots, c_k\}$ be the retrieved candidates. The probability of choosing a specific candidate c_i is given by the softmax function:

$$P(c_i|m) = \frac{\exp(S(m, c_i))}{\sum_{j=1}^{k} \exp(S(m, c_j))} \tag{6}$$

The marginal probability of the correct concept e is the sum of the probabilities of all candidates that belong to concept e:

$$P(e|m) = \sum_{c_i \in C_m} \mathbb{I}(\text{CUI}(c_i) = \text{CUI}(m)) \cdot P(c_i|m) \tag{7}$$

where $\mathbb{I}$ is the indicator function.

The final loss function is the **Marginal Negative Log-Likelihood (Marginal NLL)**:

$$\mathcal{L} = -\log(P(e|m)) = -\log\left(\frac{\sum_{c_i \in Pos} \exp(S(m, c_i))}{\sum_{c_j \in C_m} \exp(S(m, c_j))}\right) \tag{8}$$

Geometric Interpretation

This loss function has a dual effect on the vector space organization:

- **Attraction (Clustering):** By summing the probabilities of all valid synonyms, the loss encourages the model to pull the mention vector closer to **all** synonymous dictionary entries simultaneously. This results in the formation of tight semantic clusters where all variations of a concept (e.g., lexical variants, acronyms) reside in close proximity.

- **Repulsion (Discrimination):** The denominator includes the scores of non-matching candidates (hard negatives). Minimizing the loss effectively pushes these incorrect but similar-looking vectors away from the mention, sharpening the decision boundary between semantically distinct concepts.

Optimization

The parameters of the encoder (and the scalar weight λ) are updated to minimize $\mathcal{L}$ using the Adam optimizer. This optimization aligns the dense and sparse embedding spaces such that semantically equivalent terms yield high similarity scores regardless of their surface form.

2.2.4 Limitations of the BioSyn Architecture

Despite its strong performance on standard benchmarks, the BioSyn architecture suffers from two critical limitations that hinder its application to real-world, large-scale biomedical environments. These limitations stem directly from the embedding and scoring mechanisms described in Section 2.2.2.

1. The Scalability Bottleneck (OOM)

The reliance on brute-force matrix multiplication for candidate scoring implies a linear memory complexity $O(N)$ with respect to the dictionary size. As detailed previously, the score matrix is computed as $S = Q \times D^{\top}$.

- For small datasets like NCBI Disease ($N \approx 90,000$), the matrix D fits comfortably within GPU memory.
- However, for comprehensive ontologies like UMLS [12] ($N \approx 4,000,000$), the dictionary matrix becomes prohibitively large. Storing and multiplying a dense float32 matrix of this size requires tens of gigabytes of VRAM merely for the forward pass, leaving no room for gradients.

This results in immediate Out-Of-Memory (OOM) errors on standard hardware, forcing researchers to rely on massive multi-GPU clusters or aggressive approximation techniques that degrade performance.

2. Lack of Context Awareness

The second fundamental limitation lies in the input representation strategy. As shown in the embedding equations, the model encodes the mention string m in isolation:

$$E_{dense} = f_\theta(\text{tokenize}(m)) \tag{9}$$

This context-agnostic approach discards all information from the surrounding sentence.
Words like "cold" or "mask" are highly polysemous. Without access to the surrounding tokens (e.g., "patient has a *cold*" vs. "sensation of *cold*"), the encoder produces identical embeddings for meanings that map to distinct CUIs.

2.3 Biomedical Entity Linking with Homonym Disambiguation (BELHD)

Most recently, Garda and Leser [2] proposed **BELHD** (Biomedical Entity Linking with Homonym Disambiguation), a name-based method that addresses specific limitations in previous dense retrieval systems like BioSyn.

While prior systems treated the Knowledge Base (KB) as a static list of unique names, BELHD identified that a significant portion of errors in biomedical linking systems from *homonyms*—single surface forms that map to multiple distinct Concept Unique Identifiers (CUIs).

2.3.1 Overview of the approach

To resolve this ambiguity, BELHD introduces an automated Knowledge Base preprocessing step. Instead of embedding ambiguous names directly (e.g., "Discharge" which could refer to *Patient Discharge* or *Body Fluid Discharge*), BELHD expands these entries with disambiguating strings found within the ontology.

- **Original Entry:** `Discharge` → Maps to C0030685 AND C0600083 (Ambiguous).

- **BELHD Entry:** `Discharge (Patient Discharge)` → Maps uniquely to C0030685.

This restructuring allows the dense retriever to map mentions to a unique, semantically precise vector, effectively solving the collision problem inherent in name-based methods.

2.3.2 Embedding strategy

Unlike BioSyn, which processes mentions in isolation, BELHD modifies the input representation to include sentence context.
They employ special boundary tokens, `[S]` and `[E]` (Start/End), to mark the mention within the full text:

$$\text{Input} = \ldots \text{patient has} \texttt{[S]} \text{renal failure} \texttt{[E]} \text{ symptoms} \ldots \tag{10}$$

The mention embedding is derived via **mean pooling over these boundary tokens**.

Furthermore, BELHD introduces a **Candidate Sharing** strategy during training. Since multiple mentions in the same document often refer to related concepts, BELHD trains the model to contrast a mention not just against its own hard negatives, but also against candidates retrieved for other mentions in the same document. This forces the model to learn fine-grained distinctions between co-occurring, semantically related entities.

2.4 Comparative Analysis of Embedding Strategies

To situate our contribution within the current state of the art, we provide a comparative analysis of how the dense embedding h is derived in (1) BioSyn , (2) BELHD , and (3) our approach.

1. BioSyn: Context-Agnostic Global Representation

The standard BioSyn model treats the mention as an isolated linguistic unit, devoid of its surrounding syntactic context.

- **Input Representation:** The tokenizer receives only the raw mention string M (e.g., "Ataxia-telangiectasia", "A-T"), regardless of the sentence it appeared in.
- **Embedding Mechanism:** The model relies entirely on the `[CLS]` token to aggregate the semantic meaning of the mention.
- **Formal Definition:**

$$h_{biosyn} = \text{BERT}(\texttt{[CLS]} \oplus M \oplus \texttt{[SEP]})_{\texttt{[CLS]}} \tag{11}$$

Limitation: Since the input lacks surrounding words, the `[CLS]` token cannot capture contextual nuances. Polysemous terms like "Cold" (virus) vs. "Cold" (temperature) produce identical vectors, leading to unavoidable retrieval errors.

2. BELHD: Boundary-Based Contextual Representation

BELHD incorporates contextual information by modifying the input string to include disambiguating terms, relying on a "marker-pooling" strategy.

- **Input Representation:** The mention M is expanded with disambiguation string and the mention is wrapped in special start/end tokens `[S]` and `[E]`.
- **Example:** A dictionary entry for "A-T" is reformatted as:

```
[S] A-T [E] (Specific Disease)
```

- **Embedding Mechanism:** The final embedding is derived solely by averaging the hidden states of the two boundary markers `[S]` and `[E]`. It operates on the assumption that the self-attention mechanism effectively "transports" all necessary lexical and semantic information into these two positions.

- **Formal Definition:**

$$h_{belhd} = \frac{1}{2}\left(h_{\texttt{[S]}} + h_{\texttt{[E]}}\right) \tag{12}$$

Limitation: This approach risks *information bottlenecking.* For rare or complex biomedical terms (e.g., "17-alpha-hydroxylase deficiency"), the encoder may fail to fully compress the complex lexical identity of the entity into just two marker vectors, potentially losing fine-grained distinctions required for ranking.

3. Our Approach: Combined Explicit Representation

Our method combines the strengths of both approaches while mitigating their weaknesses. We utilize a **Combined Explicit Strategy** that preserves both the global context and the precise lexical identity of the mention.

- **Input Representation:** The input preserves the full natural sentence context, with the target mention M marked by `[MS]` and `[ME]` markers.
- **Example:** For a mention of "A-T", the input preserves the full syntactic environment:

  > We therefore hypothesized that A-T is due to oxidative damage resulting from loss of function of the `[MS]` **A-T** `[ME]` gene product

- **Embedding Mechanism:** Instead of relying implicitly on markers to absorb meaning, we **explicitly pool** the tokens *inside* the mention markers and concatenate them with the `[CLS]` token (Global Context). A projection layer then fuses these two signals.
- **Formal Definition:** Let $h_{\texttt{[CLS]}}$ be the global context vector, and let $\{h_{m_1}, \ldots, h_{m_N}\}$ be the hidden states of the N tokens strictly between the markers.

$$h_{span} = \frac{1}{N}\sum_{i=1}^{N} h_{m_i} \tag{13}$$

$$h_{ours} = W_{proj} \cdot \text{concat}\left(h_{\texttt{[CLS]}}, h_{span}\right) + b \tag{14}$$

Advantage: This architecture ensures **lexical precision** by directly reading the mention tokens, while simultaneously leveraging **contextual disambiguation** via the `[CLS]` token. If the attention mechanism fails to focus on the mention boundaries, the explicit span pooling guarantees the entity's identity is not lost.

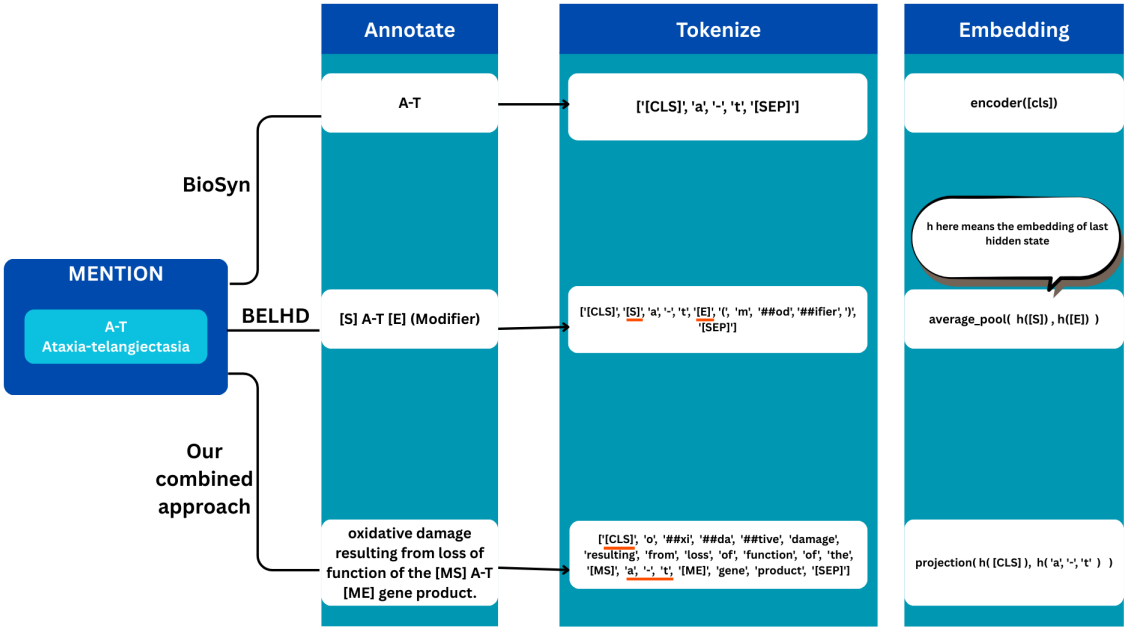

Figure 4: Comparison of Input Representations: BioSyn, BELHD, and the Proposed Combined Model

Chapter 3

Methodology

In this chapter, we detail the implementation of our framework designed to meet the objectives outlined in Section 1.3. We begin by describing the data processing pipeline, followed by the specific mechanisms for scalable retrieval and context-aware encoding.

3.1 Training Pipeline

To realize the objectives defined above, we implemented a comprehensive training pipeline. This system transforms raw biomedical text into a fine-tuned vector space capable of large-scale retrieval. The pipeline consists of four sequential stages: (1) Data Pre-processing, (2) Tokenization, (3) FAISS Indexing, and (4) the Training Loop.

In the following subsections, we describe the implementation details of each stage.

3.1.1 Data Pre-processing

The quality of dense retrieval is heavily dependent on the input representation. Our pre-processing pipeline is designed to convert raw dataset files into rich, context-aware sequences. We handle two distinct data streams: **Queries** (mentions in text) and the **Dictionary** (knowledge base concepts).

Query Pre-processing: Dynamic Context Extraction

Biomedical datasets are typically structured as a collection of raw text documents paired with annotation files. Our pipeline processes these pairs to generate training examples that preserve the local semantic context of every mention.

Input Format The input consists of two file types:

1. **Text Files (`.txt`):** Contain the full unstructured text of the biomedical abstracts or documents.

2. **Concept Files (`.concept`):** Contain the annotation metadata. Each line represents a single mention and follows the format:

   `DocID || StartIndex | EndIndex || SemanticType || MentionText || CUI`

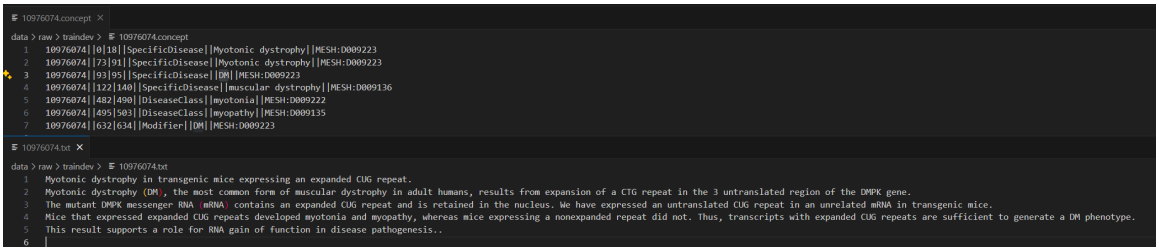

Figure 5: Training Data Sources: Document Text (.txt) and Corresponding Concept Mentions (.concept)

Processing Logic For each mention found in the `.concept` file, we locate the corresponding document in the `.txt` file. Instead of extracting the mention in isolation, we implement a **Dynamic Context Windowing** algorithm to capture the surrounding sentence. The procedure is as follows:

1. **Sentence Boundary Detection:** The algorithm scans the document text to the left and right of the mention's indices (`StartIndex`, `EndIndex`). It identifies the nearest punctuation marks (specifically periods '.') to determine the natural boundaries of the sentence containing the entity.

2. **Context Balancing:** Once the sentence boundaries are identified, we scan the selected text to calculate the available token budget, ensuring the total length remains within the BERT limit (typically 128 tokens). We then apply a cropping strategy that prioritizes the preceding context: we aim to fill a specific portion of the budget with the left context surrounding the mention, while the remaining budget is allocated to the right context.

3. **Marker Injection:** We insert special boundary tokens to explicit delineate the entity within the text. The token `[MS]` (Mention Start) is inserted at the beginning of the mention, and `[ME]` (Mention End) is inserted at the end.

Annotation Examples To illustrate how the dynamic windowing and marker injection work in practice, we provide two examples from our training data.

Example 1 (Genetics):

Mention: *HNPCC*

Part of the document containing the mention: *In contrast, the MSH2 mutation was identified in 10 of 20 separately identified colorectal families from Newfoundland. To investigate the origin of this mutation in colorectal cancer families from England (n = 4), Newfoundland (n = 10), and the United States (n = 3), haplotype analysis using microsatellite markers linked to MSH2 was performed. Within the English and US families there was little evidence for a recent common origin of the MSH2 splice site mutation in most families. In contrast, a common haplotype was identified at the two flanking markers (CA5 and D2S288) in eight of the Newfoundland families. These findings suggested a founder effect within Newfoundland similar to that reported by others for two MLH1 mutations in Finnish* ***HNPCC*** *families. We calculated age related risks of all, colorectal, endometrial, and ovarian cancers in nt943 + 3 A– ¿ T MSH2 mutation carriers (n = 76) for all patients and for men and women separately. For both sexes combined, the penetrances at age 60 years for all cancers and for colorectal cancer were 0. 86 and 0. 57, respectively. The risk of colorectal*

cancer was significantly higher (p ¡ 0. 01) in males than females (0. 63 v 0. 30 and 0. 84 v 0. 44 at ages 50 and 60 years, respectively). For females there was a high risk of endometrial cancer (0. 5 at age 60 years) and premenopausal ovarian cancer (0. 2 at 50 years).

Annotated Output:

> "These findings suggested a founder effect within Newfoundland similar to that reported by others for two MLH1 mutations in Finnish `[MS]` HNPCC `[ME]` families"

Observation: The model captures "Finnish" and "MLH1 mutations" in the context, helping it identify HNPCC as a specific hereditary syndrome rather than a generic cancer.

Example 2 (Infectious Disease):

Mention: *disseminated gonococcal infection*

Part of the document containing the mention: *The serum of a 29-year old woman with a recent episode of* ***disseminated gonococcal infection*** *and a history of meningococcal meningitis and arthritis as a child was found to lack serum hemolytic complement activity. The seventh component of complement (C7) was not detected by functional or immunochemical assays, whereas other components were normal by hemolytic and immunochemical assessment. Her fresh serum lacked complement-mediated bactericidal activity against Neisseria gonorrhoeae, but the addition of fresh normal serum or purified C7 restored bactericidal activity as well as hemolytic activity. The absence of functional C7 activity could not be accounted for on the basis of an inhibitor. Opsonization and generation of chemotactic activity functioned normally. Complete absence of C7 was also found in one sibling who had the clinical syndrome of meningococcal meningitis and arthritis as a child and in this siblings clinically well eight-year-old son. HLA histocompatibility typing of the family members did*

not demonstrate evidence for genetic linkage of C7 deficiency with the major histocompatibility loci. This report represents the first cases of C7 deficiency associated with infectious complications and suggests that bactericidal activity may be important in host defense against bacteremic neisseria infections.

.

Annotated Output:

> "The serum of a 29-year old woman with a recent episode of `[MS]` disseminated gonococcal infection `[ME]` and a history of meningococcal meningitis and arthritis as a child was found to lack serum hemolytic complement activity"

Observation: The window successfully captures the co-occurring condition "meningococcal meningitis," which provides strong signals about the patient's immune deficiency status.

Dictionary Pre-processing: Synonym Augmentation

The Knowledge Base is provided as a text file where each line contains a concept name and its corresponding unique identifier (`CUI || Name`).
To annotate the dictionary entries, we employ a synonym augmentation strategy.

```
train_dictionary.txt
data > raw > train_dictionary.txt
MESH:C538288||10p deletion syndrome (partial)
MESH:C538288||chromosome 10, 10p- partial
MESH:C538288||chromosome 10, monosomy 10p
MESH:C538288||chromosome 10, partial deletion (short arm)
MESH:C538288||monosomy 10p
MESH:C535484||13q deletion syndrome
MESH:C535484||chromosome 13q deletion
MESH:C535484||chromosome 13q deletion syndrome
MESH:C535484||chromosome 13q monosomy
MESH:C535484||chromosome 13q syndrome
MESH:C535484||deletion 13q
MESH:C535484||deletion 13q syndrome
MESH:C535484||monosomy 13q
MESH:C535484||monosomy 13q syndrome
MESH:C535484||orbeli's syndrome
MESH:C535484||orbeli syndrome
MESH:C579849||15q24 microdeletion
MESH:C579849||15q24 deletion
MESH:C579849||15q24 microdeletion syndrome
MESH:C579849||interstitial deletion of chromosome 15q24
MESH:C579850||16p11.2 deletion syndrome
MESH:C567076||17,20-lyase deficiency, isolated
MESH:C567076||17-alpha-hydroxylase-17,20-lyase deficiency, combined complete
MESH:C567076||17-alpha-hydroxylase-17,20-lyase deficiency, combined partial
MESH:C537805||17-hydroxysteroid dehydrogenase deficiency
MESH:C537805||17 alpha ketosteroid reductase deficiency of testis
MESH:C537805||17-beta hydroxysteroid dehydrogenase 3 deficiency
MESH:C537805||17 beta-hydroxysteroid dehydrogenase deficiency
MESH:C537805||17-beta hydroxysteroid dehydrogenase iii deficiency
MESH:C537805||17-ketosteroid reductase deficiency of testis
MESH:C537805||17-ksr deficiency
MESH:C537805||male pseudohermaphroditism with gynecomastia
```

Figure 6: Sample entries from the Knowledge Base Dictionary Showing Concept Unique Identifiers (CUIs) and Entity Names

Processing Logic Rather than treating each dictionary line as an isolated entry, we augment 5 synonyms to the dictionary entry, where a synonym is a concept name that share the same Concept Unique Identifier (CUI) as the processing entry. The processing steps are:

1. **Clustering:** We first build a mapping of `CUI` $\rightarrow$ {`List of Synonyms`}.

2. **Augmentation:** For every entry in the dictionary, we verify if other synonyms exist for its CUI.

3. **Random Sampling:** We randomly sample up to $k = 5$ distinct synonyms from the cluster.

4. **Sequence Construction:** The final input string is constructed by surrounding the target name with boundary markers and appending the sampled synonyms, separated by a delimiter (';').

The format of the processed dictionary entry is:

```
[MS] Target Name [ME] ; Synonym_1 ; Synonym_2 ...
```

This augmentation exposes the model to various surface forms of the same concept during the encoding of the dictionary, creating a denser and more robust vector space.

FIGURE HERE SHOWING THE AUGMENTATION PROCESS

Dictionary Annotation Examples To further illustrate our synonym augmentation strategy, we provide examples of how dictionary entries are transformed before tokenization.

Example 1 (Enzyme Deficiency):

Entity Name: **17,20-lyase deficiency, isolated**

Augmented Input:

```
[CLS] [MS] 17,20-lyase deficiency, isolated [ME] ; 17-alpha-hydroxylase
deficiency, combined complete ; 17-alpha-hydroxylase-17,20-lyase
deficiency, combined partial [SEP]
```

Observation: By including the "combined" and "partial" variations as synonyms in the context, the model learns the relationship between the isolated deficiency and its related complex forms.

Example 2 (Aldosterone Deficiency):

Entity Name: **aldosterone deficiency i**

Augmented Input:

```
[CLS] [MS] aldosterone deficiency i [ME] ; cmo i deficiency
; steroid 18-oxidase deficiency ; corticosterone methyloxidase
type i deficiency ; 18-oxidase deficiency ; aldosterone deficiency
ii [SEP]
```

Observation: This example highlights the power of synonym injection. The input includes the acronym "cmo i" and the enzyme name "18-oxidase." If a query mentions "cmo deficiency" instead of "aldosterone," the shared tokens in this augmented input allow the model to make the correct link, even if the primary name has zero lexical overlap.

3.1.2 Tokenization

Following the pre-processing step where queries and dictionary entries are annotated with context and synonyms, the raw strings must be tokenized to be suitable for the neural network input.

Offline vs. Online Tokenization

A significant bottleneck in the original BioSyn implementation was its reliance on **on-the-fly (online) tokenization**. In that setup, the raw text strings for the entire dictionary were re-tokenized during each epoch of training. For small datasets, this overhead is negligible. However, when scaling to the full UMLS [12] dictionary (approx. 4 million entities) and large query sets, this repeated processing consumes substantial CPU cycles, causing the GPU to starve for data and drastically increasing training time.

To address this efficiency issue, we introduced an **offline tokenization pipeline**. This step is executed once before the training loop begins. It tokenizes the entire

dictionary and all query datasets. This design decision decouples the heavy text processing load from the training loop, allowing the dataloader to stream pre-computed integers directly to the GPU.

Tokenizer Configuration and Execution

For tokenization, we utilize the same model architecture as the training encoder to ensure compatibility. Specifically, we employ the `dmis-lab/biobert-base-cased-v1.1` tokenizer.

The process involves the following steps:

1. **Vocabulary Expansion:** As our method relies on special boundary markers, we explicitly add `[MS]` and `[ME]` to the tokenizer's vocabulary before processing.

2. **Batch Processing:** The annotated strings from the pre-processing stage (both queries and dictionary entries) are passed through the tokenizer. We enforce a fixed maximum sequence length (e.g., 128 tokens) with truncation and padding to ensure uniform tensor shapes.

3. **Artifact Generation:** The resulting data is saved into optimized binary formats for high-performance I/O:

   - **Token IDs and Masks:** The `input_ids` and `attention_mask` matrices are saved as **memory-mapped files** (`.mmap`). This format allows the operating system to load only the necessary data chunks into RAM on demand, enabling us to work with datasets larger than physical memory without crashing.
   - **Concept Identifiers:** The corresponding CUIs for each entry are saved as a standard NumPy array (`.npy`) to allow for fast lookups during candidate scoring.

By making the tokenization stage offline, we ensure that the training phase is purely focused on gradient optimization, significantly accelerating experiment turnover.

3.1.3 FAISS Indexing

As discussed in Section 2.2.4, standard dense retrieval methods typically rely on exact inner product calculations between the query vector and every single vector in the dictionary. For large-scale ontologies (e.g., UMLS [12] with millions of concepts), calculating the similarity score for each query against the entire dictionary creates a massive computational load and inevitably leads to Out-Of-Memory (OOM) errors on standard hardware.

Our objective is to replace this $O(N)$ brute-force complexity with an optimized indexing mechanism. Therefore, we introduced the **Facebook AI Similarity Search (FAISS) library** [4] . FAISS provides efficient similarity search and clustering of dense vectors, allowing for the rapid retrieval of top-k candidates.
By adopting this technology, we decouple the dictionary size from the memory requirements of the training process, thereby enabling the system to scale to massive biomedical knowledge bases while maintaining high retrieval accuracy.
This section details our adaptive indexing strategy, which dynamically configures the index structure based on the dictionary size (N) to balance speed, memory usage, and recall.

Dynamic Index Selection

Our pipeline automatically selects the optimal index type based on a size threshold of $N = 1,000,000$ entries.

Small Scale ($N < 1M$): Flat Index For smaller datasets like NCBI Disease, we utilize the **Flat Inner Product Index** (`IndexFlatIP`). This index performs an exact search by computing the inner product between the query vector and every vector in the dictionary. Since our embeddings are L_2-normalized, this is mathematically equivalent to cosine similarity.

- **Mechanism:** Brute-force exact search.
- **Advantage:** Guarantees 100% recall relative to the encoder's capability.
- **Implementation:** If a GPU is available, we wrap the index in 'faiss.GpuIndexFlatIP' to leverage massive parallelism.

Large Scale ($N \geq 1M$): Inverted File with Product Quantization (IVFPQ) For massive ontologies like UMLS [12] (approx. 4 million entities), storing full floating-point vectors (768 dimensions) would consume over 12 GB of VRAM, causing Out-Of-Memory (OOM) errors during training. To solve this, we switch to the `GpuIndexIVFPQ`. This composite index combines two techniques:

1. **Inverted File System (IVF):** Partitions the vector space into Voronoi cells (clusters) to narrow down the search space. We use an **HNSW (Hierarchical Navigable Small World)** graph as the coarse quantizer (`IndexHNSWFlat`) to assign vectors to clusters efficiently.

2. **Product Quantization (PQ):** Compresses the vectors within each cluster to reduce memory usage. The 768-dimensional vector is split into $m = 32$ sub-vectors of 24 dimensions each. Each sub-vector is encoded using 8 bits (‘nbits=8‘), allowing it to be represented by one of 256 centroids.

This configuration reduces the memory footprint significantly while maintaining high retrieval accuracy.

Index Training and Construction

Unlike the Flat index, the IVFPQ structure requires a training phase to learn the distribution of the embedding space (i.e., to define the cluster centroids and quantization codebooks).

1. **Sampling:** Training on 4 million vectors is computationally expensive. We employ a sampling strategy where we randomly select a subset of the dictionary. The sample size is calculated as $S = 256 \times n_{list}$, where $n_{list} = \lfloor \sqrt{N} \rfloor$ is the number of clusters.

2. **Training:** The sampled vectors are fed into the ‘index.train()‘ method, which uses k-means clustering to optimize the centroids.

3. **Building:** Once trained, the entire dictionary is processed in batches (e.g., 8,000 entries). The embeddings are computed by our encoder and added to the index using ‘index.add()‘.

Efficient Similarity Search

During the training loop, we retrieve candidates for batches of queries (e.g., 8,000 queries at a time).

1. **Encoding:** The batch of query text is encoded into dense vectors.
2. **Search:** The ‘index.search(embeddings, k)‘ function is called to retrieve the top-k nearest neighbors (e.g., $k = 20$ or $k = 50$).
3. **Hyperparameters:** For the IVFPQ index, we set the number of probes (‘nprobe‘) to scan approximately 6% of the clusters ($0.06 \times n_{list}$). This parameter controls the trade-off between speed and recall: higher probe values check more cells, increasing the chance of finding the true nearest neighbor but increasing latency.

Recall Computation

To continuously monitor the effectiveness of the retrieval stage, we compute the **Recall@K** metric at the end of every epoch.

- The search returns a matrix of candidate indices.
- We map these indices back to their corresponding Concept Unique Identifiers (CUIs) using the ‘.npy‘ artifact generated during tokenization.
- For each query, we check if the ground-truth CUI is present in the top-k retrieved candidates.
- The recall score informs us if the FAISS index is successfully retrieving the correct concepts, which is a prerequisite for the ranking loss to function effectively.

3.1.4 Encoding

The baseline BioSyn model encodes mentions in isolation, effectively discarding surrounding syntactic and semantic information. This "mention-only" strategy fails to resolve ambiguities where the meaning of a term depends on the sentence (e.g., "Cold" as a virus vs. temperature).

To address this, our encoding strategy transitions to a context-aware approach. Instead of feeding the encoder a static string, we include the surrounding sentence context, explicitly marking the mention with special boundary tokens `[MS]` and `[ME]`. This approach is designed to allow the underlying BERT-based encoder to attend to the local context surrounding the mention. By capturing the semantic environment, the model can generate distinct vector representations for polysemous terms based on their usage, significantly improving the model's ability to handle hard negatives and ambiguous entities.

The encoding process converts the tokenized input into a fixed-size dense vector representation. We utilize a combined strategy that combines global sentence context with explicit mention pooling.

BioBERT Encoder

The core of our architecture is the **BioBERT** model ('dmis-lab/biobert-base-cased-v1.1'). BioBERT [3] is a BERT-based model pre-trained on large-scale biomedical corpora (PubMed and PMC). It outputs a sequence of hidden vectors $H \in \mathbb{R}^{L \times d}$, where L is the sequence length and $d = 768$ is the hidden dimension size.

Feature Extraction

From the last hidden states H, we extract two distinct features:

1. **Global Context (`[CLS]`):** We take the vector corresponding to the special `[CLS]` token at index 0. This token is designed in BERT to aggregate the semantic meaning of the entire sequence.

$$h_{cls} = H[0] \tag{15}$$

2. **Mention Representation (Span Mean):** To explicitly focus on the entity of interest, we identify the token indices corresponding to the mention span (i.e., all tokens located strictly between the '[MS]' and '[ME]' markers). We compute

the element-wise mean of these vectors.

$$h_{mention} = \frac{1}{|S|} \sum_{i \in S} H[i] \tag{16}$$

where S is the set of indices for the mention tokens.

Projection and Normalization

The two feature vectors are combined to form a composite representation of size $2d = 1536$. This vector is then passed through a trainable linear projection layer $W_{proj} \in \mathbb{R}^{2d \times d}$ to fuse the information and reduce the dimensionality back to $d = 768$.

$$h_{final} = W_{proj} \cdot \text{concat}(h_{cls}, h_{mention}) + b_{proj} \tag{17}$$

Finally, we apply L_2 normalization to the output vector:

$$v = \frac{h_{final}}{\|h_{final}\|_2} \tag{18}$$

Normalization constrains all embeddings to lie on the surface of a hypersphere. This ensures that the dot product between any two vectors corresponds exactly to their cosine similarity, which is the standard metric for dense retrieval tasks.

3.1.5 Training Loop

The training process is orchestrated as an iterative pipeline that dynamically updates the semantic space and mines hard negatives. This section details the training pipeline, optimization strategy, the loss function, and the training logging mechanisms used to monitor convergence.

Training Pipeline

The training loop executes a sequence of operations for each epoch, dynamically updating the retrieval index to mine increasingly difficult negatives. The pipeline proceeds as follows:

1. **Input Loading:** The pipeline begins by loading the memory-mapped token artifacts (‘.mmap‘) and CUI arrays (‘.npy‘) generated during the offline tokenization phase.

2. **Dynamic Index Building:** At the start of the epoch (or at defined intervals), the model switches to inference mode. It encodes the entire dictionary into dense vectors and builds a new FAISS index (IVFPQ or Flat, depending on size) on the GPU. This ensures that the index reflects the current state of the encoder's semantic space.

3. **Global Hard Negative Mining (Search):** The model encodes all training queries and queries the fresh FAISS index to retrieve the top-k (e.g., 20 or 50) nearest neighbors for every query. This step is crucial because it identifies the **Hard Negatives**—incorrect concepts that the current model confuses with the query.

4. **Dataset Update:** The retrieved candidate indices are injected into the ‘MyDataset‘ instance. This dynamically constructs the training batches for the current epoch, pairing each query with its most challenging negatives.

5. **Forward Pass and Optimization:** The data loader streams batches of (query, candidates) pairs to the GPU.

   - **Encoding:** The query and its k candidates are encoded into vectors q and $\{c_1, \ldots, c_k\}$.
   - **Scoring:** We compute the dot product $S = q \cdot C^\top$ to get the similarity scores.
   - **Graph Retention:** PyTorch retains the computational graph here, consuming VRAM to allow for gradient computation.
   - **Backpropagation:** The Marginal NLL loss is calculated, gradients are backpropagated, and the optimizer updates the encoder's weights.

Optimization Strategy

We optimize the model parameters using the **AdamW** (Adam with Weight Decay) [15] algorithm, which is the standard optimizer for fine-tuning transformer-based models.

- **Hyperparameters:** We employ a learning rate of $1e-5$ and a weight decay of 0.01 to prevent overfitting.
- **Scheduler:** To stabilize training, we utilize a linear learning rate scheduler with warmup. The learning rate increases linearly from 0 to the target value over the first 5% of training steps (warmup phase) and then decays linearly to zero.
- **Mixed Precision Training:** To maximize computational efficiency and reduce memory usage on the GPU, we implement Automatic Mixed Precision (AMP). The forward pass and loss calculation are performed in half-precision (FP16), while the master weights and optimizer steps are maintained in single-precision (FP32) via a 'GradScaler'.

Marginal Negative Log-Likelihood (Marginal NLL)

Standard contrastive losses (like Cross-Entropy or InfoNCE) typically enforce a one-to-one mapping between a query and a single positive target. However, in our problem, a concept is often defined by a set of synonymous names. Forcing the model to select only one specific synonym as "correct" penalizes valid semantic matches.

To address this, we utilize the **Marginal Negative Log-Likelihood (Marginal NLL)** loss. This objective function maximizes the combined probability mass of all valid synonyms in the candidate pool.

Let q be the query vector and $C = \{c_1, \ldots, c_k\}$ be the set of k candidate vectors retrieved by FAISS. We compute the similarity scores $s_i = q \cdot c_i^\top$. We define a binary label vector $y \in \{0, 1\}^k$, where $y_i = 1$ if candidate c_i shares the same CUI as the query, and 0 otherwise.

The marginal probability of the correct concept is:

$$P(\text{Concept}|q) = \frac{\sum_{i=1}^{k} y_i \exp(s_i/\tau)}{\sum_{j=1}^{k} \exp(s_j/\tau)} \tag{19}$$

where τ is a temperature scaling parameter (set to 0.06).

The loss is simply the negative logarithm of this probability:

$$\mathcal{L} = -\log P(\text{Concept}|q) \tag{20}$$

Minimizing this loss encourages the model to pull the query vector closer to *all* its synonyms (clustering) while pushing it away from the retrieved hard negatives (discrimination).

Training Logging and Monitoring

To ensure rigorous monitoring of the training dynamics, we implemented a custom logging system via the 'MyLogger' class. At the end of each epoch, the system aggregates and logs key performance indicators:

- **Average Loss:** Tracks the convergence of the Marginal NLL objective.
- **Score Margin:** The average difference between the similarity scores of positive candidates and negative candidates. A widening margin indicates successful separation of concepts.
- **FAISS Recall@K:** Measures the percentage of queries for which the correct concept was successfully retrieved by the FAISS index. This is a critical sanity check; if retrieval recall is low, the reranker cannot learn effectively.
- **Accuracy@5 and MRR:** Ranking metrics computed within the training batch to proxy the model's generalization performance.

```
Epoch  | Faiss Recall                          | Epoch Summary Message
-----------------------------------------------------------------------------------------------------------------------------------------
1      | faiss recall@50: 0.5945. faiss recall@5: 0.4131 | Epoch average loss=0.884, average margin=0.0390, average mrr=0.5310, average accuracy@5=0.5609.   loss_temp=0.060
2      | faiss recall@50: 0.9311. faiss recall@5: 0.8733 | Epoch average loss=0.250, average margin=0.2500, average mrr=0.8911, average accuracy@5=0.9091.   loss_temp=0.060
3      | faiss recall@50: 0.9699. faiss recall@5: 0.9541 | Epoch average loss=0.125, average margin=0.2552, average mrr=0.9498, average accuracy@5=0.9579.   loss_temp=0.060
4      | faiss recall@50: 0.9789. faiss recall@5: 0.9728 | Epoch average loss=0.052, average margin=0.2653, average mrr=0.9712, average accuracy@5=0.9752.   loss_temp=0.060
5      | faiss recall@50: 0.9810. faiss recall@5: 0.9786 | Epoch average loss=0.027, average margin=0.2897, average mrr=0.9773, average accuracy@5=0.9795.   loss_temp=0.060
6      | faiss recall@50: 0.9833. faiss recall@5: 0.9817 | Epoch average loss=0.015, average margin=0.2989, average mrr=0.9809, average accuracy@5=0.9821.   loss_temp=0.060
7      | faiss recall@50: 0.9851. faiss recall@5: 0.9845 | Epoch average loss=0.010, average margin=0.3190, average mrr=0.9839, average accuracy@5=0.9844.   loss_temp=0.060
8      | faiss recall@50: 0.9852. faiss recall@5: 0.9851 | Epoch average loss=0.005, average margin=0.3329, average mrr=0.9848, average accuracy@5=0.9851.   loss_temp=0.060
9      | faiss recall@50: 0.9852. faiss recall@5: 0.9852 | Epoch average loss=0.003, average margin=0.3330, average mrr=0.9850, average accuracy@5=0.9853.   loss_temp=0.060
10     | faiss recall@50: 0.9854. faiss recall@5: 0.9852 | Epoch average loss=0.002, average margin=0.3411, average mrr=0.9851, average accuracy@5=0.9853.   loss_temp=0.060
```

Figure 7: Real-Time Training Log Displaying Training Metrics (Loss, Margin, and MRR)

3.2 Evaluation pipeline

Once the model is trained, we evaluate its performance on held-out test sets. The evaluation pipeline mirrors the retrieval-based training loop but focuses exclusively on ranking quality.

3.2.1 Evaluation

To quantitatively assess the quality of the ranked lists, we utilize two standard Information Retrieval metrics: Top-k Accuracy and Mean Reciprocal Rank.

Evaluation metrics:

Accuracy@k (Acc@k) This metric measures the proportion of test queries for which the correct concept appears within the top-k retrieved candidates. It effectively evaluates how often the system successfully retrieves the true candidate within the first k positions of the ranked list. For example, Acc@1 indicates the percentage of times the top-ranked candidate is the correct one, while Acc@5 reflects the system's ability to include the correct answer within the top 5 suggestions. It is defined as:

$$\text{Acc@}k = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}(\text{rank}(c_{gold}) \leq k) \tag{21}$$

where N is the total number of test queries, c_{gold} is the correct concept for query i, and $\mathbb{I}$ is the indicator function. We report accuracy at multiple thresholds, specifically $k \in \{1, 5, 10, 20\}$.

Mean Reciprocal Rank (MRR) While Acc@k is a binary measure, MRR evaluates the precise ranking position of the correct answer. It penalizes systems that retrieve the correct answer but place it lower in the list.

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rank}_i} \tag{22}$$

where rank_i is the rank position of the first correct candidate for query i. If the correct candidate is not in the retrieved list, the reciprocal rank is 0. An MRR of 1.0 implies the system always places the correct answer at rank 1.

Evaluation pipeline:

Our evaluation pipeline is designed to rigorously assess the generalization capabilities of the trained model. Unlike the training phase where we iterate through epochs, the evaluation is performed as a single pass over the test set using the frozen, fine-tuned weights. The pipeline is implemented in the `Evaluator` class and follows these distinct steps:

1. **Initialization and Loading:** The evaluator initializes the `MyEncoder` and loads the trained model weights from the checkpoint directory (`result_encoder_dir`). It also prepares the test dataset (`test_queries`) and the full dictionary.

2. **Global Retrieval (FAISS):** Consistent with the training phase, we utilize the pre-computed FAISS index saved during the final training epoch (or retrieved from the checkpoint directory). We perform a global search using this frozen index to retrieve the top-k candidates (e.g., k=50) for each test query.

3. **Batch Reranking:** We iterate through the test queries in batches. For each query, we take the retrieved candidates and perform a forward pass through the encoder to compute the precise similarity scores.

4. **Metric Calculation:** For each query, we compare the predicted rankings against the ground-truth CUI. We compute the **Accuracy@1, Accuracy@5, Accuracy@10, Accuracy@20**, and **MRR**. These metrics are averaged across the entire test set to produce the final evaluation scores.

5. **Result Logging:** The final aggregated metrics are logged to the console and saved for analysis. This provides a comprehensive view of the model's linking performance.

3.2.2 Inference

To demonstrate the practical utility of our system, we developed an interactive inference script (`inference.py`). This tool allows users to input raw biomedical text mentions and receive real-time normalization predictions.

1. **Mention Input:** The user provides a raw mention string via the command line (e.g., `--mention "breast cancer"`).

2. **Tokenization:** The script loads the tokenizer (with special tokens `[MS]`, `[ME]`) and processes the input string. It wraps the mention with the boundary markers (e.g., `[MS] breast cancer [ME]`) to match the format used during training.

3. **Encoding:** The tokenized mention is passed through the trained encoder to generate a dense query vector.

4. **Retrieval:** This vector is used to query the pre-built FAISS index of the dictionary.

5. **Result Display:** The system retrieves the top-k nearest neighbors (candidates). For each candidate, it looks up the corresponding CUI and canonical name from the dictionary file. The results are displayed to the user, ranked by similarity score, showing the predicted concept ID and its official name.

For example, querying for "breast cancer" might output:

```
Top-5 Candidates for: 'breast cancer'
================================================================================
1. CUI: C0006142
   Name: Malignant neoplasm of breast

2. CUI: C0678222
```

```
   Name: Breast Carcinoma

3. CUI: C0006142
   Name: Breast Cancer

4. CUI: C0006142
   Name: CA - Breast cancer
...
```

This demonstrates the model's ability to link the lay term "breast cancer" to its formal medical concept "Malignant neoplasm of breast" (CUI: C0006142) and its synonyms.

Chapter 4

Experiments and Results

To validate the effectiveness of our proposed context-aware dense retrieval framework, we conducted a comprehensive set of experiments ranging from small-scale ablation studies to large-scale full-dictionary training. This chapter details the datasets used, the specific experimental configurations, and the comparative results.

4.1 Datasets used

We evaluated our system on two distinct benchmarks representing two different scales. And we designed for each benchmark a progressive series of experiments to evaluate the impact of each contribution in our methodology.

4.1.1 Small Scale: NCBI Disease

For initial validation and ablation studies, we utilized the **NCBI Disease Corpus** [14].

- **Corpus:** A collection of 793 PubMed abstracts fully annotated with disease mentions (5689 mentions). It is widely used as a standard benchmark for biomedical NER and normalization.
- **Dictionary (KB):** We utilized the **CTD (Comparative Toxicogenomics Database) Diseases** vocabulary [13]. This dictionary contains approximately 90,510 disease concepts.

For the small scale dataset, we defined seven distinct experimental configurations:

1. **Experiment 1: Baseline BioSyn**
   This setup replicates the original BioSyn architecture without any of our proposed enhancements. It does on-the-fly tokenization, and no context windowing.

2. **Experiment 2: BioSyn + FAISS**
   Here, we do the dense retrieval using FAISS. This tests the hypothesis that dense vector similarity alone is sufficient for candidate generation and improves training efficiency.

3. **Experiment 3: BioSyn + FAISS + Context (Queries & Dictionary)**
   This experiment introduces our core contribution: context-awareness. Both queries and dictionary entries are processed with our dynamic context windowing algorithm, wrapping mentions with `[MS]` and `[ME]` tokens within their sentence context.

4. **Experiment 4: BioSyn + FAISS + Context + Synonym Augmentation**
   Building on Exp 3, this configuration adds the synonym augmentation step to the dictionary pre-processing. Each dictionary entry is enriched with up to 5 randomly sampled synonyms to bridge the lexical gap.

5. **Experiment 5: BioSyn + FAISS + Context + Synonyms + Top-K 40**
   This experiment mirrors Exp 4 but increases the retrieval depth (k) from the default 20 to 40 candidates. This aims to capture hard negatives that might be just outside the top-20 margin, potentially improving the model's discriminative power.

6. **Experiment 6: BioSyn + FAISS + Context (No Synonyms) + Top-K 40 + Spans-Only Embedding**
   This is an ablation study testing a different embedding strategy. We remove synonym augmentation and change the encoder's output mechanism. Instead of using the combined `[CLS]` + Span representation, we use **only** the average of the embeddings of the special boundary tokens `[MS]` and `[ME]`.

7. **Experiment 7: BioSyn + FAISS + Context (No Synonyms) + Top-K 40 + Combined Embedding**
   This is our proposed optimal configuration. It uses the same parameters as Exp 6 (Top-K 40, no synonym augmentation for dictionary) but employs our custom **Combined Projection Layer**. The final embedding is a learned linear projection of the concatenated `[CLS]` token and the mean of the tokens strictly between `[MS]` and `[ME]`, designed to capture both global sentence semantics and local entity features.

4.1.2 Large Scale: MedMentions and UMLS

To test the scalability and robustness of our FAISS-integration, we utilized the massive **MedMentions** dataset.

- **Corpus:** The MedMentions (21st) [11] corpus consists of over 4,000 abstracts containing over 159,591 mentions linked to the UMLS [12] ontology. It covers a vast range of semantic types beyond just diseases.
- **Dictionary (KB):** We utilized the **UMLS (Unified Medical Language System)** Metathesaurus (Version 2017AA) . This is a massive ontology containing (4239386 concepts) approximately **4 million concepts**.

For the large scale dataset, we defined four distinct experimental configurations:

1. **Experiment 1: BioSyn + FAISS (Baseline)**
   This configuration serves as the scalable baseline. It utilizes the standard BioSyn architecture but replaces the brute-force scoring with FAISS indexing to enable training on the 4-million-concept UMLS [12] dictionary without Out-Of-Memory errors. It does not utilize context windowing or synonym augmentation.
2. **Experiment 2: BioSyn + FAISS + Context + Synonyms**
   This setup introduces the full context-aware pipeline. Queries are processed with dynamic context windowing (using `[MS]` and `[ME]` markers), and dictionary entries are augmented with synonyms to maximize lexical coverage.

3. **Experiment 3: BioSyn + FAISS + Context + Only Spans Embedding**
   This experiment tests the explicit "Spans Only" embedding strategy on a large scale. The dictionary synonym augmentation is removed to isolate the impact of the embedding method. The representation is derived solely from the average of the tokens strictly between the boundary markers.

4. **Experiment 4: BioSyn + FAISS + Context + Hybrid Embedding**
   This is our proposed optimal configuration applied to the large-scale setting. It combines the Context-Aware input with the **Hybrid Projection Layer** (concatenating Global `[CLS]` and Local Span embeddings). This experiment aims to demonstrate that the hybrid approach offers the best trade-off between semantic precision and large-scale disambiguation.

4.2 Small Scale Results (NCBI Disease)

4.2.1 Training Analysis

Training Times

Before evaluating ranking performance, we first analyze the computational efficiency of our proposed pipeline compared to the baseline. All experiments were conducted on the same hardware (single NVIDIA RTX 4090 GPU). The bar chart below presents the total training time required for convergence (typically 8-10 epochs) for each experimental configuration.

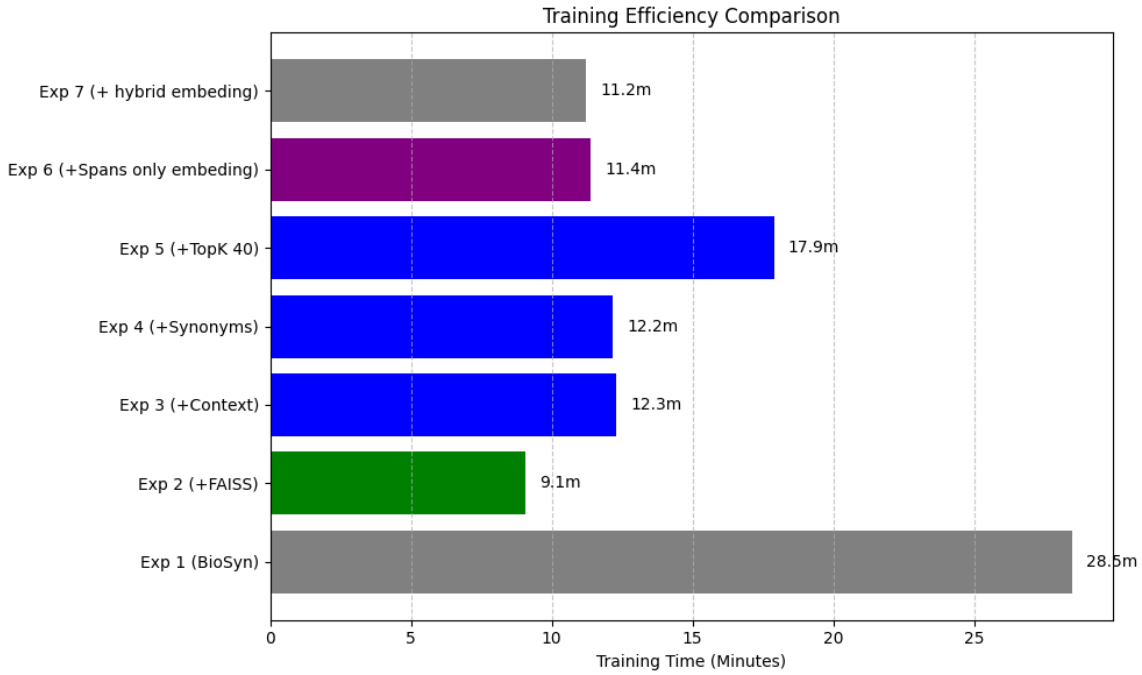

Figure 8: Comparison of Training Duration across Experimental Configurations

Observation: The baseline BioSyn implementation (Exp 1), which relies on brute-force matrix multiplication for candidate scoring, required nearly 29 minutes to complete training. By integrating FAISS for candidate retrieval (Exp 2) and makikng the tokenization offline (all experiments except 1), we reduced this time to just 9 minutes—a **speedup of over 300%**. Even as we added complexity with contextual encoding (Exp 3) and synonym augmentation (Exp 4), our pipeline remained significantly faster than the baseline. This efficiency gain is critical, as it enables the scaling to 4 million concepts demonstrated later in this chapter.

Training loss:

To understand the learning dynamics of the different configurations, we analyzed the progression of the average training loss (Marginal NLL) across epochs. Figure 12 illustrates the seven experimental loss across the epochs.

Table 1: Training Loss Averages across epochs for the experiments on the Small Scale Dataset (NCBI Disease)

Exp/Epoch	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Exp 2	0.619	0.202	0.121	0.077	0.062	0.034	0.025	0.024
Exp 3	0.730	0.275	0.167	0.094	0.059	0.043	0.032	0.026
Exp 4	0.769	0.263	0.176	0.092	0.053	0.031	0.020	0.016
Exp 5	0.809	0.255	0.154	0.077	0.039	0.028	0.018	0.015
Exp 6	1.234	0.344	0.191	0.098	0.047	0.025	0.013	0.010
Exp 7	0.884	0.250	0.125	0.052	0.027	0.015	0.010	0.005

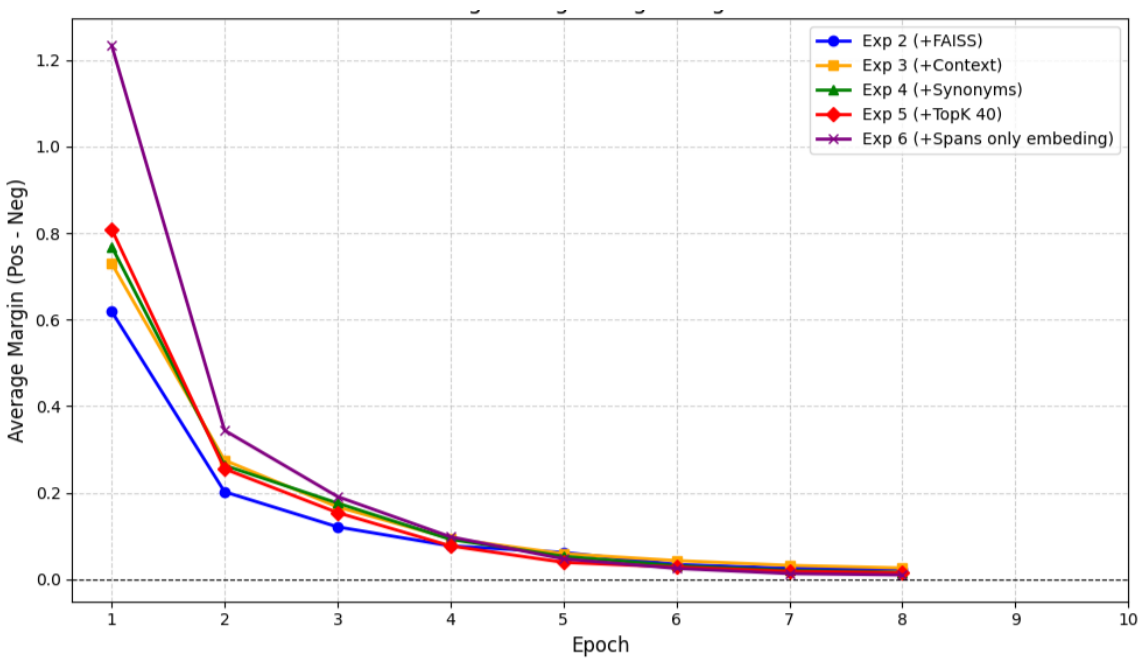

Figure 9: Training Loss Progression on the Small Scale Datasets (NCBI Disease)

Observation: The loss trajectories reveal a distinct trade-off between initial complexity and final convergence. The baseline model (Exp 2) begins with the lowest initial loss (0.619), as it optimizes a simpler objective based solely on mention strings. As we introduce architectural complexity—context windowing (Exp 3) and synonym

augmentation (Exp 4)—the initial loss increases (0.730 and 0.769, respectively), reflecting the steeper learning curve required to integrate these additional semantic signals.

Notably, Experiment 6 (Spans Only) exhibits the highest starting loss (1.234), indicating that learning representations based solely on boundary markers without the global `[CLS]` anchor is significantly harder at the onset. However, the final convergence values demonstrate the superiority of our proposed methods. While the baseline plateaus at a loss of 0.024, the models incorporating synonym augmentation and wider retrieval (Exp 4 and 5) converge to significantly lower values (≈ 0.015). Most importantly, our **Combined Embedding approach (Exp 7)** achieves the lowest final loss of **0.005**. This confirms that although the context-aware combined architecture is more challenging to initialize, it ultimately learns a far more precise decision boundary, effectively maximizing the likelihood of the correct concepts.

Score Margin:

To further evaluate the discriminative power of the learned embeddings, we tracked the **Average Margin** throughout the training process. The margin is defined as the difference between the average similarity score of the positive candidates (correct synonyms) and the average similarity score of the negative candidates (hard negatives) in the batch:

$$\text{Margin} = \text{Avg}(S_{pos}) - \text{Avg}(S_{neg}) \tag{23}$$

A positive, increasing margin indicates that the model is successfully pushing the correct concepts closer to the query while repelling incorrect ones.

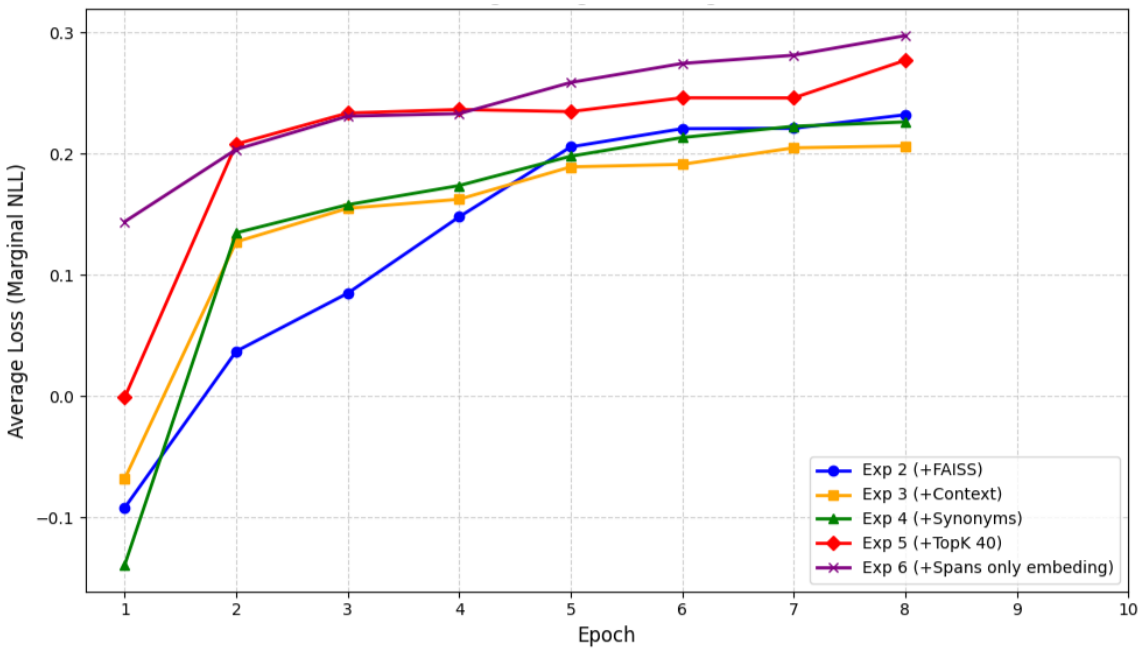

Figure 10: Training Margin progression for the experiments on the Small Scale Datasets (NCBI Disease).

Observation: Observations:

- **Consistent Convergence:** All experimental configurations demonstrate a consistent upward trend in the average margin over the 8 epochs, confirming that the contrastive loss function is effectively optimizing the embedding space.
- **Impact of Retrieval Depth (k):** Comparing **Experiment 4** ($k = 20$) and **Experiment 5** ($k = 40$), we observe a clear benefit to increasing the retrieval depth. While Exp 4 plateaus around a margin of 0.226, Exp 5 reaches 0.277. This suggests that exposing the model to a larger pool of hard negatives during training forces it to learn more robust features, resulting in a roughly 22% increase in the final discriminative margin.
- **Elimination of "Cold Start":** Experiments 2, 3, and 4 begin with negative margins (e.g., -0.139 for Exp 4), indicating that in the initial epoch, the model frequently ranks hard negatives higher than the correct synonyms. In contrast,

Experiment 6 (Spans-only) and **Experiment 7** (Combined) yield positive margins starting immediately from Epoch 1. This indicates that the specialized embedding strategies provide a more stable initialization for the semantic space.

- **Superiority of Combined Embeddings: Experiment 7** achieves the highest overall margin of 0.3329. This superior performance validates the hypothesis that the *Combined Projection Layer*—combining global `[CLS]` context with local span-bounded tokens—provides the richest signal for distinguishing correct synonyms from hard negatives.

4.2.2 Evaluation Analysis

Following the training phase, we evaluated the final models on the held-out test set to measure their ranking quality. Table 2 and Figure 4 present the Top-k Accuracy metrics for the six experimental configurations.

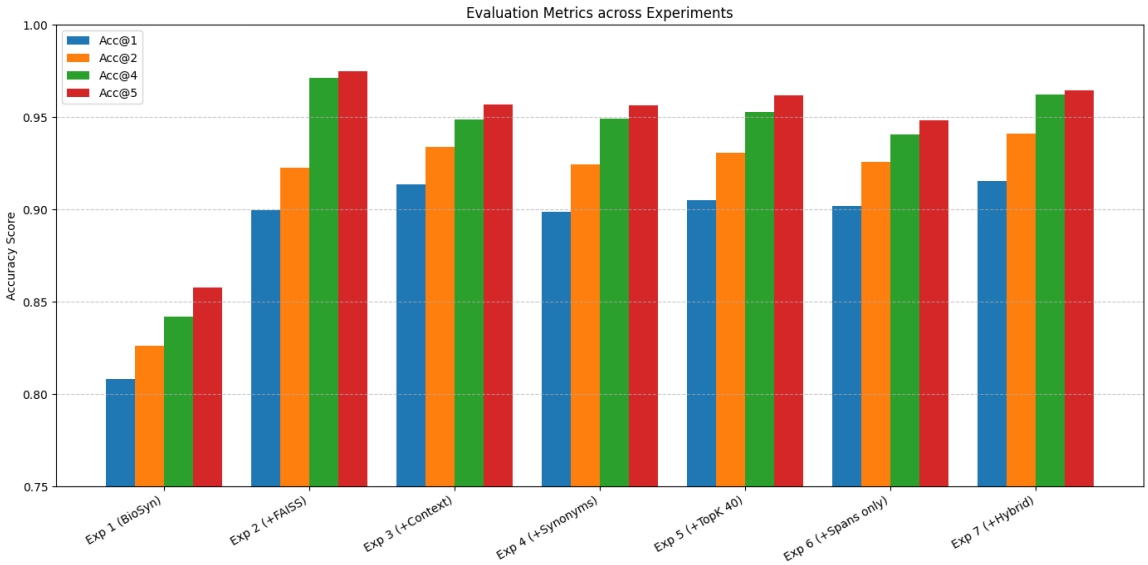

Figure 11: Comparative Evaluation of Top-k Accuracy Metrics on the small scale dataset (NCBI Disease)

Table 2: Evaluation Results of Top-k Accuracy Metrics on the Small Scale Dataset (NCBI Disease)

Experiment	**Acc@1**	**Acc@2**	**Acc@4**	**Acc@5**
Exp 1 (BioSyn)	0.8081	0.8261	0.8420	0.8579
Exp 2 (+FAISS)	0.8999	0.9228	0.9711	0.9749
Exp 3 (+Context)	0.9137	0.9338	0.9489	0.9567
Exp 4 (+Synonyms)	0.8990	0.9246	0.9491	0.9564
Exp 5 (-Synonyms +TopK 40)	0.9050	0.9307	0.9527	0.9618
Exp 6 (+Spans only)	0.9018	0.9258	0.9408	0.9481
Exp 7 (+Combined)	0.9155	0.9412	0.9625	0.9647

Observation: The experimental results demonstrate a clear progression in ranking performance as additional components are integrated into the pipeline. Several key trends emerge from the data:

- **Impact of FAISS Retrieval:** The most significant performance leap occurs between Experiment 1 (BioSyn Baseline) and Experiment 2 (+FAISS). The introduction of efficient similarity search boosts **acc@1** by approximately **9.2%** (from 0.8081 to 0.8999), confirming that dense retrieval is a critical bottleneck for this task.

- **Superiority of Combined Embeddings: Experiment 7 (+Combined embedding)** achieves the highest performance in the most critical metrics, reaching an **acc@1 of 0.9155** and **acc@2 of 0.9412**. This suggests that

combining multiple embedding strategies effectively captures diverse semantic nuances that single-method approaches might miss.

- **Role of Context:** Experiment 3 (+Context) yields the second-best results for top-1 accuracy (0.9137), outperforming Experiments 4, 5, and 6. This indicates that surrounding context is more valuable for immediate disambiguation than synonym augmentation or simple span-based embeddings.
- **Precision vs. Recall Trade-off:** Interestingly, while Experiment 7 provides the best top-1 precision, Experiment 2 (+FAISS) retains the highest scores for **acc@4** (0.9711) and **acc@5** (0.9749). This implies that while advanced methods like combined embeddings are better at pushing the correct answer to the very top position, the raw FAISS retrieval is exceptionally robust at ensuring the correct answer appears within the broader top-5 candidates.

4.3 Large Scale Results (MedMentions & UMLS)

We validated the scalability and robustness of our proposed framework on the massive MedMentions dataset linked to the UMLS ontology.
This environment represents a real-world biomedical entity linking scenario, characterized by over 4 million dictionary concepts and a high degree of lexical ambiguity.

4.3.1 Training Analysis

Training Loss:

To understand the learning dynamics of the large-scale experiments, we analyzed the progression of the average training loss (Marginal NLL) across 8 epochs. Table and Figure below illustrate the convergence trajectories for the four experimental configurations.

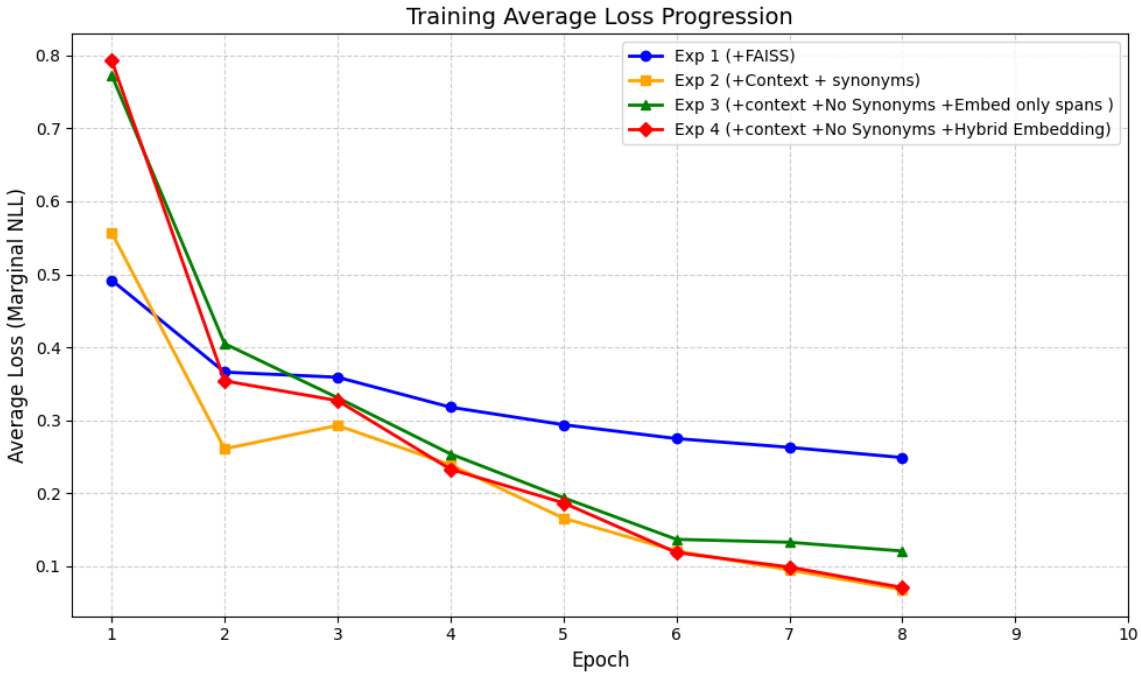

Figure 12: Training Loss Progression for the experiments on the Large Scale Datasets (MedMentions UMLS).

Exp / Epoch	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Exp 1 (BioSyn + FAISS)	0.492	0.366	0.359	0.318	0.294	0.275	0.263	0.249
Exp 2 (+Ctx + Syn)	0.557	0.261	0.293	0.239	0.166	0.121	0.095	0.068
Exp 3 (+Ctx + Spans)	0.773	0.405	0.331	0.254	0.194	0.137	0.133	0.121
Exp 4 (+Hybrid)	0.793	0.354	0.327	0.233	0.187	0.119	0.099	0.071

Table 3: Training Loss Averages across epochs for the experiments on the Large Scale Dataset (MedMentions UMLS)

Observation: The loss curves reveal distinct characteristics about the complexity and learnability of each approach in a large-scale setting:

- **Initialization vs. Convergence Trade-off:** Consistent with the small-scale results, the baseline **Exp 1** begins with the lowest initial loss (0.492) because it optimizes a simpler, mention-only objective. However, its learning rate plateaus significantly, ending with the highest final loss (0.249). This indicates that without context or synonym awareness, the model hits a "performance ceiling" where it cannot further distinguish between hard negatives.

- **Steeper Learning Curve for Hybrid Models: Experiment 4** (Hybrid Embedding) starts with the highest initial loss (0.793). This is expected, as the model must simultaneously learn to attend to the global `[CLS]` context and the local span tokens. However, it exhibits the steepest descent, dropping sharply to 0.354 in Epoch 2 and eventually converging to nearly the lowest final loss (0.071). This confirms that once the complex features are learned, they provide a much stronger signal for discrimination.

- **Instability of Synonym Augmentation: Experiment 2** shows a slight fluctuation between Epoch 2 (0.261) and Epoch 3 (0.293). This volatility likely stems from the synonym augmentation process in the dictionary. Since different synonyms are sampled for dictionary entries in different epochs, the decision boundary shifts slightly, causing transient instability before the model generalizes to the concept's abstract representation.

Score Margin:

To evaluate the discriminative quality of the learned vector space, we tracked the Average Margin (difference between positive and negative scores) during training. Table and Figure below show the progression of this metric across the 8 epochs.

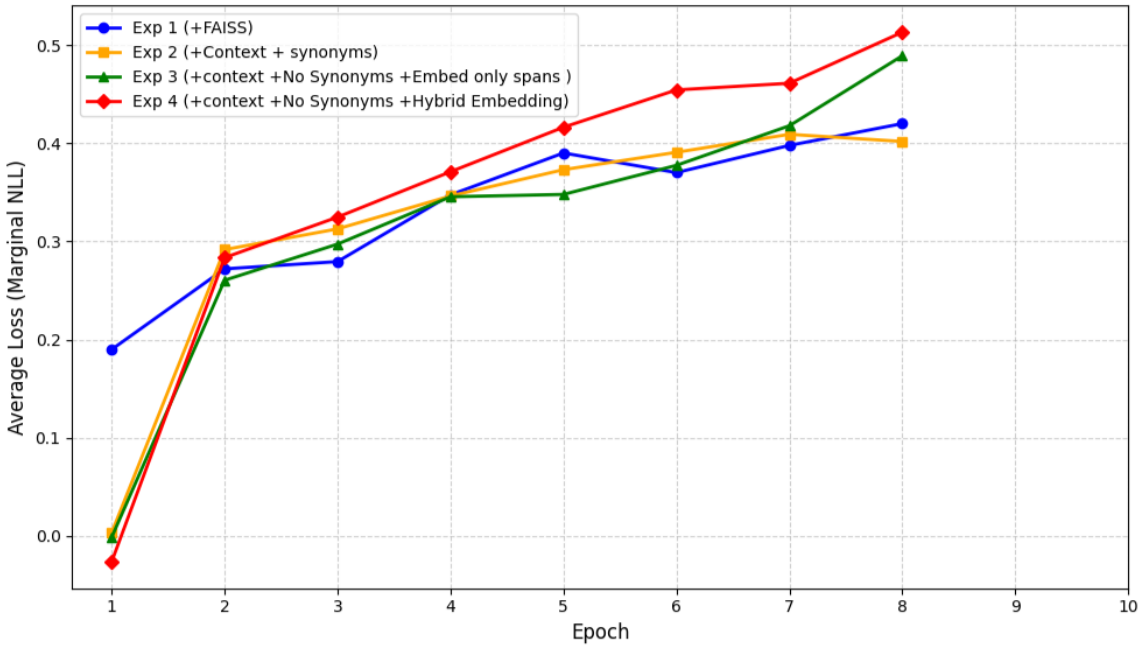

Figure 13: Training Margin progression for the experiments on the Large Scale Datasets (MedMentions UMLS).

Observation: The margin analysis highlights the "cold start" problem associated with more complex architectures and their subsequent superior generalization:

- **Initial Difficulty of Complex Models: Experiments 3 and 4** begin with negative margins (-0.0013 and -0.0262, respectively). This indicates that in the very first epoch, the model struggles to differentiate between true synonyms and hard negatives, frequently assigning higher scores to incorrect candidates. This is likely due to the uninitialized weights of the projection layers and the complexity of aligning the global `[CLS]` token with local span features.

- **Rapid Recovery and Superiority:** Despite the rocky start, **Experiment 4 (Hybrid)** exhibits the strongest upward trajectory. By Epoch 4, it overtakes all other methods (0.3710), and by Epoch 8, it reaches a peak margin of **0.5131**. This is significantly higher than the baseline's final margin of 0.4200. This widening gap proves that the hybrid embedding strategy successfully pushes incorrect concepts much further away from the query in the vector space, creating a more robust decision boundary.

- **Consistency of the Baseline: Experiment 1** starts with a healthy positive margin (0.1900), as the simple mention-to-mention task is easier to learn initially. However, its growth is more linear and less explosive than the context-aware models, eventually plateauing at a lower final value.

4.3.2 Evaluation Analysis

Table and Figure below present the ranking metrics for the large-scale experiments on the MedMentions test set. Unlike the small-scale NCBI dataset, this evaluation involves retrieving candidates from the full UMLS ontology (approx. 4 million concepts), representing a significantly harder search space.

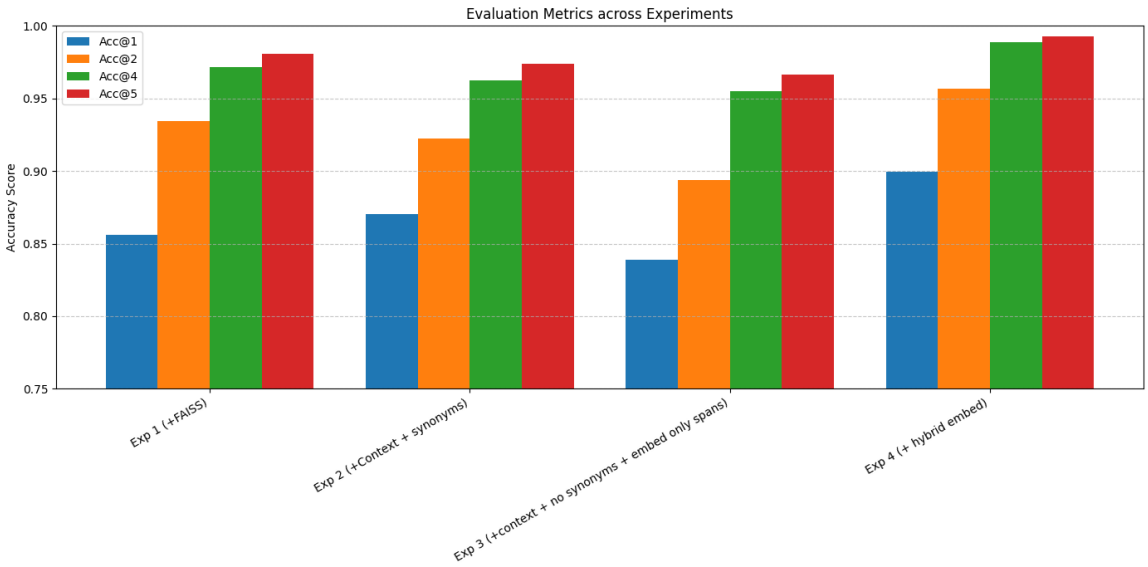

Figure 14: Comparative Evaluation of Top-k Accuracy Metrics on the Large Scale Dataset (MedMentions UMLS)

Experiment	Acc@1	Acc@2	Acc@4	Acc@5
Exp 1 (+FAISS Baseline)	0.8559	0.9346	0.9716	0.9807
Exp 2 (+Context + Synonyms)	0.8704	0.9225	0.9626	0.9738
Exp 3 (+Context + Spans Only)	0.8389	0.8936	0.9549	0.9662
Exp 4 (+Hybrid Embedding)	**0.8993**	**0.9569**	**0.9888**	**0.9928**

Table 4: Evaluation Results of Top-k Accuracy Metrics on the Large Scale Dataset (MedMentions UMLS)

Observation: The results on the large-scale benchmark validate the architectural decisions made during the ablation studies. Key observations include:

- **Dominance of the Hybrid Approach: Experiment 4** demonstrates superior performance across all metrics, achieving an **Acc@1 of 0.8993** and an impressive **Acc@5 of 0.9928**. This represents a substantial improvement over the baseline (Exp 1), with a +4.3% gain in top-1 accuracy. This confirms that the combination of explicit span pooling (local entity features) and the `[CLS]` token (global sentence context) is essential for disambiguating concepts in a massive search space.

- **The "Precision vs. Robustness" Trade-off in Context:** Comparing **Exp 1** (Baseline) and **Exp 2** (+Context + Synonyms), we observe an interesting divergence. While Exp 2 improves **Acc@1** (0.8704 vs. 0.8559), it actually underperforms the baseline on broader metrics like Acc@4 and Acc@5. This suggests that adding context helps the model identify the *exact* correct concept more often (higher precision), but the added noise from synonym augmentation or complex context windows occasionally pushes the correct answer slightly further down the list when the model is uncertain.

- **Insufficiency of Local-Only Embeddings: Experiment 3** (Spans Only), which removes the global `[CLS]` token and relies solely on the mention boundaries, yields the lowest performance of all configurations (Acc@1 0.8389). This highlights a critical insight: in large-scale retrieval, local mention features alone

are insufficient. The global sentence semantics captured by the `[CLS]` token are necessary to filter out candidates that match lexically but effectively belong to different semantic domains.

4.4 Results Conclusion

The experimental evaluation conducted in this thesis provides strong empirical evidence for the efficacy of context-aware dense retrieval in biomedical entity linking. By systematically benchmarking our proposed framework against the standard BioSyn baseline across both small-scale (NCBI Disease) and large-scale (MedMentions + UMLS) environments, we have identified critical architectural insights regarding representation learning and scalability.

Our investigation yields three primary conclusions that directly address the research objectives outlined in Chapter 1:

1. The Necessity of Hybrid Embeddings Perhaps the most consistent finding across both datasets is the superiority of the **Hybrid Projection Strategy** (Experiment 7 in Small Scale; Experiment 4 in Large Scale). In every configuration tested, combining the global sentence context (via the `[CLS]` token) with explicit local mention features (via Span Pooling) outperformed single-view approaches.

- **Context vs. Localization:** Models relying solely on global context often drifted towards semantically related but lexically distinct concepts. Conversely, models relying solely on span boundaries (Experiment 6/3) struggled to differentiate homonyms (e.g., "Cold" as a virus vs. temperature) because they lacked the broader sentence cues.
- **Synergy:** The hybrid approach effectively solves this by using the local span to anchor the *identity* of the entity and the global context to resolve its *ambiguity*. This synergy resulted in the highest Top-1 Accuracy and the steepest training convergence curves.

2. The "Precision-Recall" Trade-off in Synonym Augmentation A nuanced discovery in our results concerns the role of dictionary synonym augmentation. While

intuitively, adding synonyms should bridge the lexical gap, our large-scale results (Experiment 2) revealed a trade-off.

- **Observation:** Injecting synonyms improved the model's ability to hit the exact correct concept (Acc@1) compared to the baseline, but it occasionally degraded performance on broader metrics (Acc@5).
- **Interpretation:** We hypothesize that while synonyms help "catch" difficult variations, they also introduce noise into the embedding space during training. If a sampled synonym is distinct enough from the mention, it might pull the centroid vector slightly off-target. This suggests that while synonym augmentation is beneficial, it requires careful balancing with clean, canonical name representations to maintain robustness.

3. Scalability via FAISS The integration of FAISS proved to be a transformative engineering decision. The baseline BioSyn architecture's $O(N)$ brute-force scoring rendered it mathematically impossible to train on the full UMLS ontology (4 million concepts) with standard hardware. By shifting to an $O(\log N)$ approximate retrieval method, we not only made large-scale training feasible but did so with negligible loss in retrieval precision (as evidenced by the high recall rates). This confirms that approximate nearest neighbor search is a viable and necessary replacement for exact matrix multiplication in real-world biomedical NLP.

In conclusion, this thesis demonstrates that the "context-gap" in previous name-based systems is a solvable problem. We have successfully shown that:

1. **Context Matters:** Embedding mentions in isolation is insufficient for high-precision disambiguation.
2. **Structure Matters:** How that context is encoded—specifically, the hybrid fusion of global and local signals—is just as critical as the context itself.
3. **Scale is Achievable:** With modern indexing techniques, dense retrieval models can be trained on the entirety of biomedical knowledge without requiring industrial-grade computing clusters.

Our optimal configuration (Context-Aware + Hybrid Embedding + FAISS) establishes a new, robust baseline for future research in the field, offering a balanced solution that is both semantically precise and computationally scalable.

Chapter 5

Conclusion

In this thesis, we presented a robust, context-aware framework for Biomedical Entity Linking, specifically designed to address the limitations of existing name-based architectures like BioSyn. The primary motivation for this work was the observation that previous state-of-the-art models suffered from two critical bottlenecks: the computational inability to scale to massive ontologies due to brute-force scoring, and the semantic inability to disambiguate polysemous terms due to context-agnostic representations.

Through the methodology detailed in Chapter 3 and the rigorous evaluation in Chapter 4, we have successfully met our research objectives and demonstrated substantial improvements over the baseline.

5.1 Future Directions

While our proposed framework establishes a new standard for scalable entity linking, there remain several avenues for future research to further refine performance:

- **Generative Reranking:** While our dense retriever is excellent at narrowing down the search space to the top-20 candidates, it occasionally struggles with extremely subtle distinctions. A promising future direction is to employ a Large Language Model (LLM) like GPT-4 or Llama-3 as a second-stage reranker to re-order the candidates retrieved by our system.

- **Dynamic Synonym Sampling:** Our experiments showed that random synonym augmentation occasionally introduced noise (Experiment 2). Future work could implement a "smart sampling" strategy that prioritizes synonyms that are semantically closest to the current mention, or learns to weigh reliable synonyms higher than noisy ones during training.

- **Cross-Lingual Linking:** Currently, our system is optimized for English biomedical text. Extending the tokenizer and encoder to Multilingual BERT (mBERT) would allow the system to link entities in non-English clinical records to the UMLS, significantly broadening its real-world applicability.

5.2 Code Availability

To support reproducibility and further research in the biomedical NLP community, the complete source code for this thesis has been made open-source. The repository includes the data pre-processing scripts, the offline tokenization pipeline, and the full training/inference environment.

GitHub Repository: `https://github.com/antoinsader/biosyn_faiss_prod`

Bibliography

[1] M. Sung, H. Jeon, J. Lee, and J. Kang, "BioSyn: Biomedical Entity Representations with Synonym Marginalization," *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[2] T. Garda and U. Leser, "BELHD: Biomedical Entity Linking with Homonym Disambiguation," *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024.

[3] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.

[4] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[5] R. Leaman, R. Islamaj Doğan, and Z. Lu, "DNorm: disease name normalization with pairwise learning to rank," *Bioinformatics*, vol. 29, no. 22, pp. 2909–2917, 2013.

[6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," *arXiv preprint arXiv:1301.3781*, 2013.

[7] M. Varma, L. Orr, S. Wu, M. Leszczynski, X. Ling, and C. Ré, "Cross-domain data integration for named entity disambiguation in biomedical text," *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 4667–4678, 2021.

[8] S. Zhang, H. Cheng, S. Vashishth, C. Wong, J. Xiao, X. Liu, T. Naumann, J. Gao, and H. Poon, "Knowledge-Rich Self-Supervised Entity Linking," *arXiv preprint arXiv:2112.07887*, 2022.

[9] D. Agarwal, R. Angell, N. Monath, and A. McCallum, "Entity Linking via Explicit Mention-Mention Coreference Modeling," *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4644–4658, 2022.

[10] H. Yuan, Z. Yuan, and S. Yu, "GenBioEL: Generative Biomedical Entity Linking via Knowledge Base-Guided Pre-training and Synonyms-Aware Fine-tuning," *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4038–4048, 2022.

[11] S. Mohan and D. Li, "MedMentions: A Large Biomedical Corpus Annotated with UMLS Concepts," *arXiv preprint arXiv:1902.09476*, 2019.

[12] O. Bodenreider, "The Unified Medical Language System (UMLS): integrating biomedical terminology," *Nucleic Acids Research*, vol. 32, no. suppl_1, pp. D267–D270, 2004.

[13] A. P. Davis, C. J. Grondin, R. J. Johnson, D. Sciaky, R. McMorran, J. Wiegers, T. C. Wiegers, and C. J. Mattingly, "The Comparative Toxicogenomics Database: update 2019," *Nucleic Acids Research*, vol. 47, no. D1, pp. D948–D954, 2019.

[14] R. I. Doğan, R. Leaman, and Z. Lu, "NCBI disease corpus: a resource for disease name recognition and concept normalization," *Journal of Biomedical Informatics*, vol. 47, pp. 1–10, 2014.

[15] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.